From Multistage Stochastic Programming to Markov Decision Process

Yiping Fang, Professor

Chair on Risk and Resilience of Complex Systems,

Laboratoire Génie Industriel, CentraleSupélec, Université Paris Saclay

Yiping.fang@centralesupelec.fr

Outline

CentraleSupélec

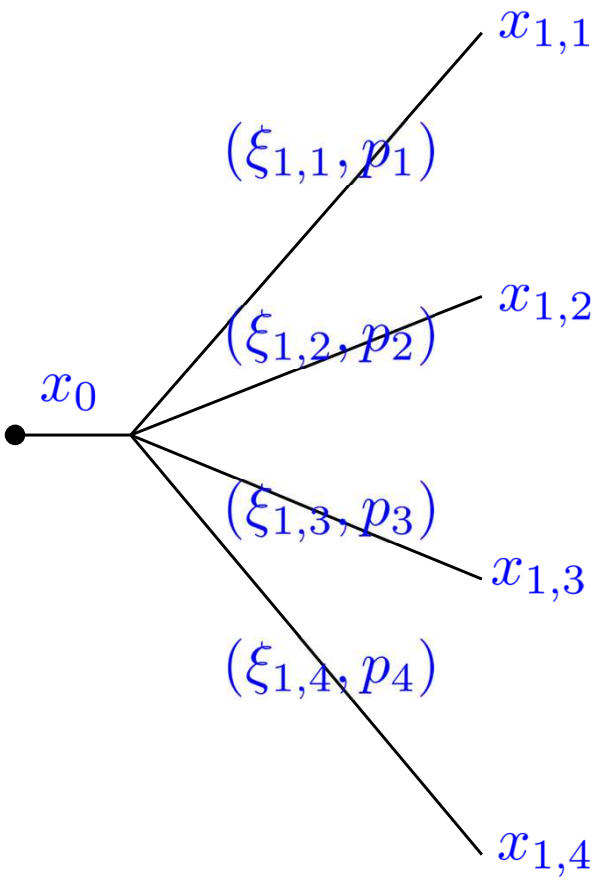
- Multistage stochastic programming
- A different perspective: Markov decision processes

  $\rightarrow$ modeling elements

  $\rightarrow$ solving MDP: value iteration and policy iteration

  $\rightarrow$ dual perspective: linear programming for MDP

Where do we come from: two-stage SP



- We take decisions in two-stages

$$x_0 \rightsquigarrow \xi_1 \rightsquigarrow x_1$$

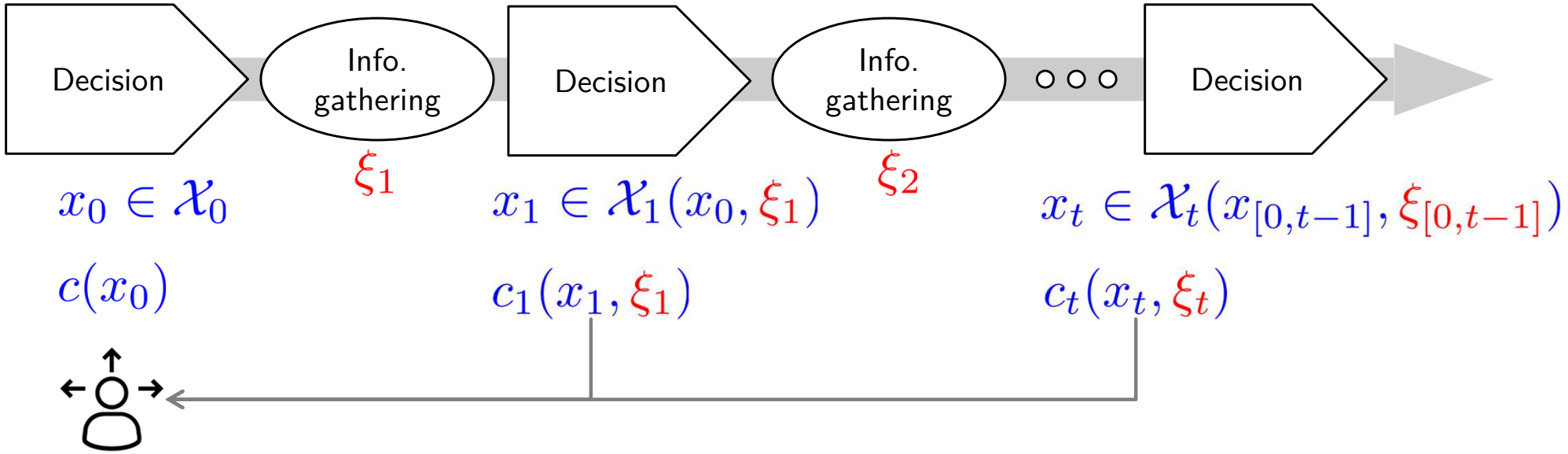
- A scenario tree is a graphical representation of a stochastic decision process

- On a scenario tree, it means solving the extensive formulation (linear):

$$\min_{x_0, x_{1,s}} \quad c_0^\top x_0 + \sum_{s \in \mathcal{S}} p_s [c_s^\top x_{1,s}]$$

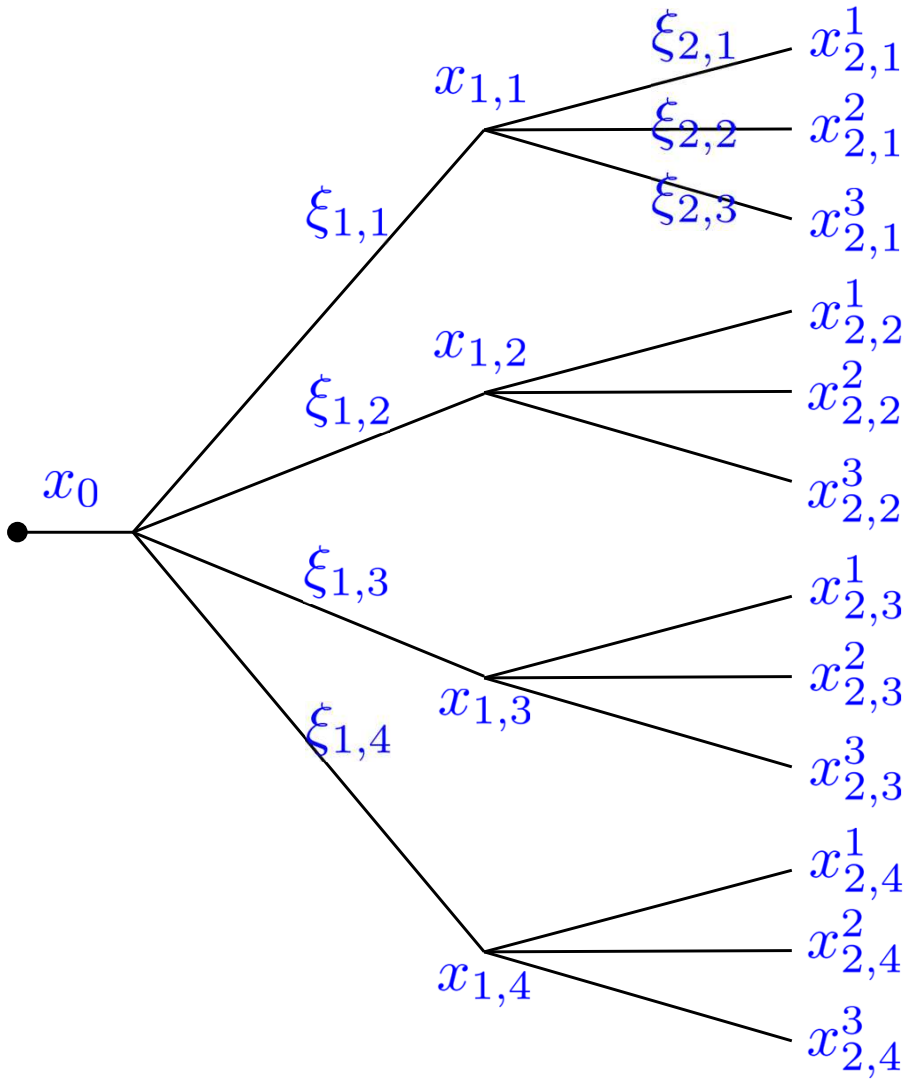
$$\text{s.t.} \quad x_0 \in \mathcal{X}_0, x_{1,s} \in \mathcal{X}_1(x_0, \xi_{1,s})$$

Generalize to multistage



- Ubiquitous
- Highly challenging, modeling and computationally
- Exogenous uncertainty: uncertainty realizations are not influenced by decisions

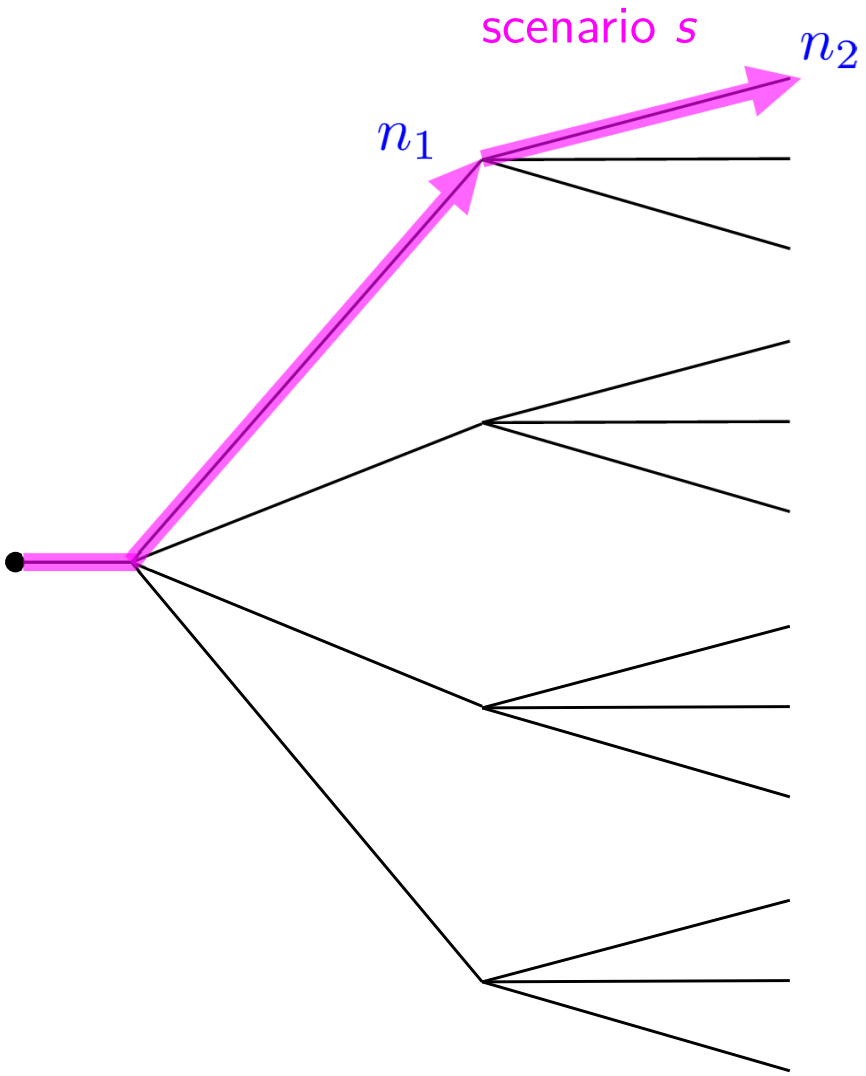
Generalize to multistage



- We take decisions in T stages
$$x_0 \rightsquigarrow \xi_1 \rightsquigarrow x_1 \rightsquigarrow \cdots \rightsquigarrow \xi_T \rightsquigarrow x_T$$
- Can be represented on a tree $\mathcal{T}$, where a node n of depth t represent a realization of
$$(\xi_1, \cdots, \xi_t) : p_n$$
- Then, the extensive form where we minimize expect total cost reads
$$\min_{\{x_n\}_{n\in\mathcal{T}}} \sum_{n\in\mathcal{T}} p_n c_n(x_n)$$

Splitted scenario formulation

CentraleSupélec

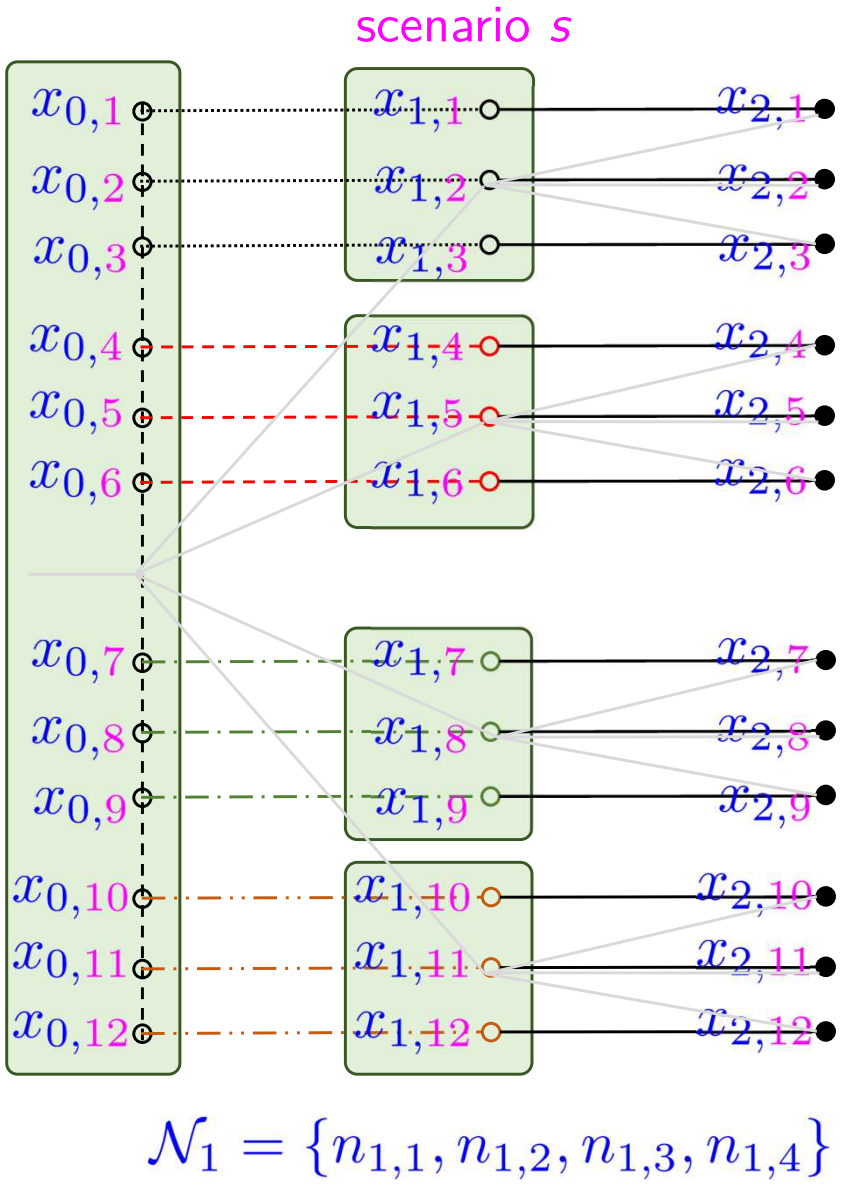


- Consider a tree of depth T. A scenario $\{s\} := (n_1, n_2, ..., n_T)$ uniquely defined by its last element (leaf) $p_s = p_{n_T}$
- The splitted scenario formulation reads

$$\min_{\{x_n\}_{n\in\mathcal{T}}} \quad \sum_{n\in\mathcal{T}} p_n c_n(x_n)$$

$$= \sum_{s\in S} p_s \sum_{n\in N_s} c_n(x_n)$$

Splitted scenario formulation



$$\mathcal{N}_1 = \{n_{1,1}, n_{1,2}, n_{1,3}, n_{1,4}\}$$

- The splitted scenario formulation

$$\min_{\{x_{t,s}\}_{s\in S, t\in\{0,\dots,T\}}} \sum_{s\in S} p_s \sum_{t=0}^{T} c_{t,s}(x_{t,s})$$

$$\text{s.t} \quad x_{t,s} = x_{t,s'},$$

$$\forall t, n \in \mathcal{N}_t, \{s\} \cap \{s'\} \ni n$$

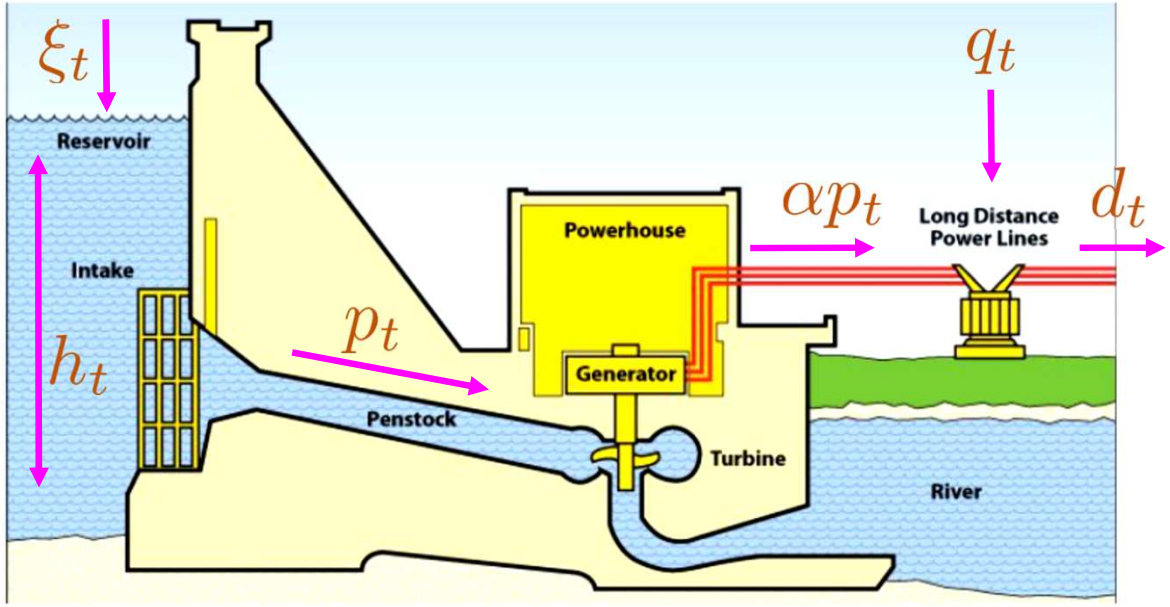
where $\mathcal{N}_t$ is the set of nodes of depth t

- Nonanticipativity: at time t, decisions are taken only knowing the past realizations of the uncertainty

Example: hydro power planning

CentraleSupélec

- How much hydro power to generate in each period to satisfy demand with minimum expected cost?



$$\min_{p_t, q_t} \quad \sum_t c_t^e q_t + c^p p_t$$

$$\text{s.t.} \quad h_t = h_{t-1} + \xi_t - p_{t-1}, \forall t$$

$$\alpha p_t + q_t = d_t, \forall t$$

$$0 \leq p_t \leq h_t \leq h^{max}, \forall t$$

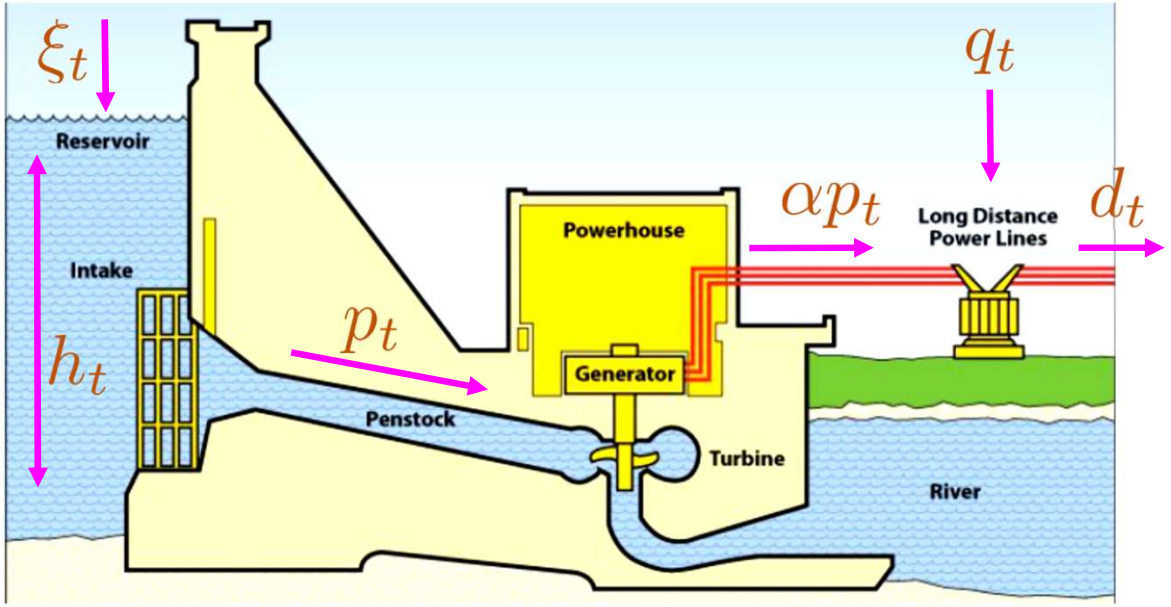
$$p_t, q_t \geq 0, \forall t$$

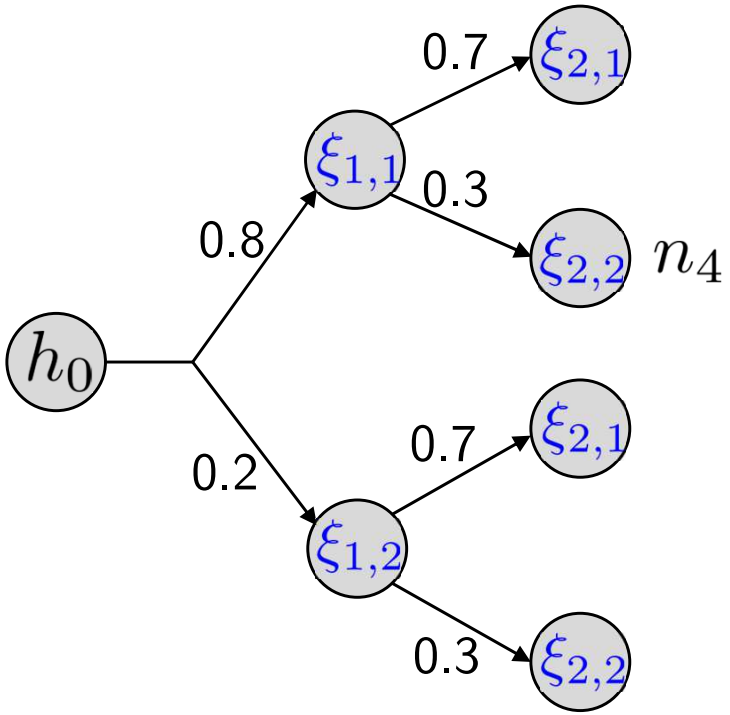
- Generation cost: $c^p p_t$
- Buying remote electricity cost: $c_t^e q_t$
- Stochastic inflow ξ_t

Example: hydro power planning

CentraleSupélec

- Stochastic inflows ξ_t follow the scenario tree
- Extensive form?
- Splitted scenario form?



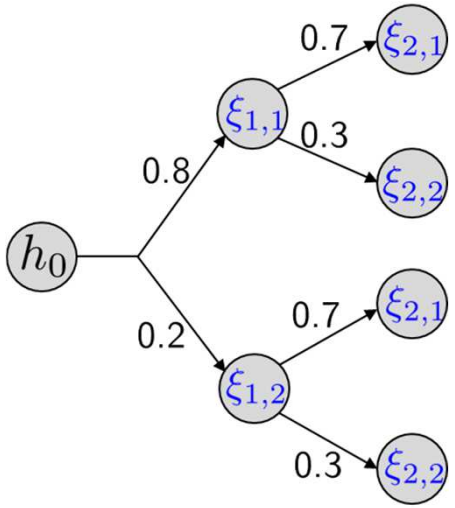


$$p_{s=2} = p_{n_4} = 0.8 \cdot 0.3 = 0.24$$

Example: hydro power planning

CentraleSupélec

- Extensive form



Node	Label	pa	p_n
n_0	root	$\emptyset$	1.00
n_1	$\xi_{1,1}$	n_0	0.80
n_2	$\xi_{1,2}$	n_0	0.20
n_3	$\xi_{1,1} \to \xi_{2,1}$	n_1	$0.80 \cdot 0.7 = 0.56$
n_4	$\xi_{1,1} \to \xi_{2,2}$	n_1	$0.80 \cdot 0.3 = 0.24$
n_5	$\xi_{1,2} \to \xi_{2,1}$	n_2	$0.20 \cdot 0.7 = 0.14$
n_6	$\xi_{1,2} \to \xi_{2,2}$	n_2	$0.20 \cdot 0.3 = 0.06$

$$\min_{\{p_n, q_n, h_n\}_{n \in \mathcal{T}}} \quad \sum_{n \in \mathcal{T}} p_n (c_n^e q_n + c^p p_n)$$

$$\text{s.t.} \quad h_n = h_{\text{pa}(n)} + \xi_n - p_{\text{pa}(n)}, \forall n \neq n_0$$

$$\alpha p_n + q_n = d_n, \forall n \in \mathcal{T}$$

$$0 \leq p_n \leq h_n \leq h^{max}, \forall n \in \mathcal{T}$$

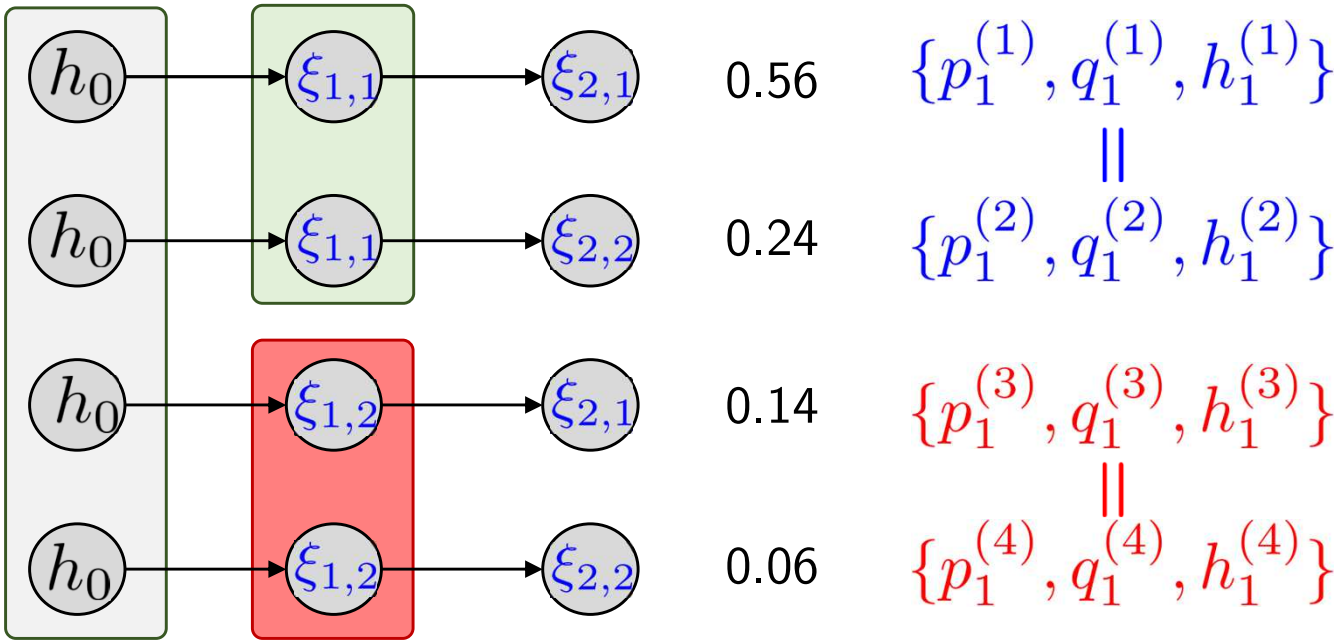
$$p_n, q_n \geq 0, \forall n \in \mathcal{T}$$

Example: hydro power planning

CentraleSupélec

- Splitted scenario form

Scenario s	Path of (ξ_1, ξ_2)	Probability p_s
$s^{(1)}$	$(\xi_{1,1},\ \xi_{2,1})$	$0.8 \cdot 0.7 = 0.56$
$s^{(2)}$	$(\xi_{1,1},\ \xi_{2,2})$	$0.8 \cdot 0.3 = 0.24$
$s^{(3)}$	$(\xi_{1,2},\ \xi_{2,1})$	$0.2 \cdot 0.7 = 0.14$
$s^{(4)}$	$(\xi_{1,2},\ \xi_{2,2})$	$0.2 \cdot 0.3 = 0.06$



Solving MSP

CentraleSupélec

- The splitted scenario formulation (although LP) is usually too large to be solved as one monolithic problem.
- **Decomposition:**

  $\rightarrow$ Stage-wise decomposition: nested Benders

  $\rightarrow$ Stochastic Dual Dynamic Programming (state-of-the-art for large-scale multistage)

- The complexity grows exponentially with the number of depth (stage) T

Outline

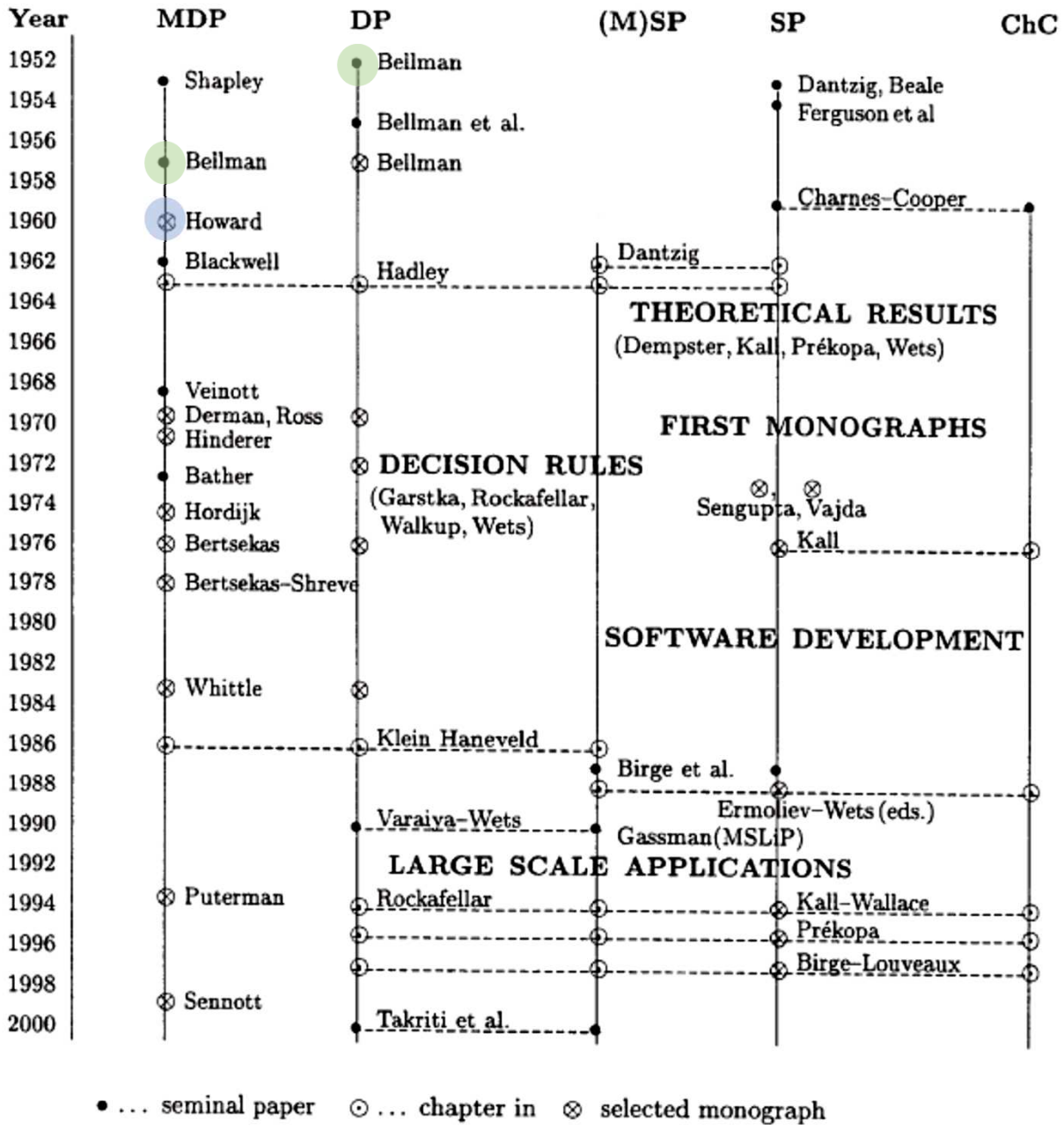
- Multistage stochastic programming
- A different perspective: Markov decision processes

  $\rightarrow$ modeling elements

  $\rightarrow$ solving MDP: value iteration and policy iteration

  $\rightarrow$ dual perspective: linear programming for MDP

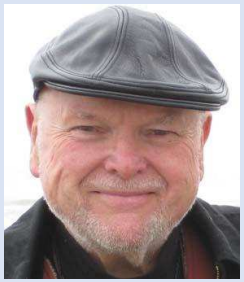
History and connection



Richard Bellman

On the theory of dynamic programming (1952)

A Markovian Decision Process (1957)



Ronald A. Howard

Dynamic programming and Markov Processes, 1960 (Book)

MDP: Markov decision processes

DP: Dynamic programming

(M)SP: (Multistage) Stochastic programming

ChC: Chance-constraint

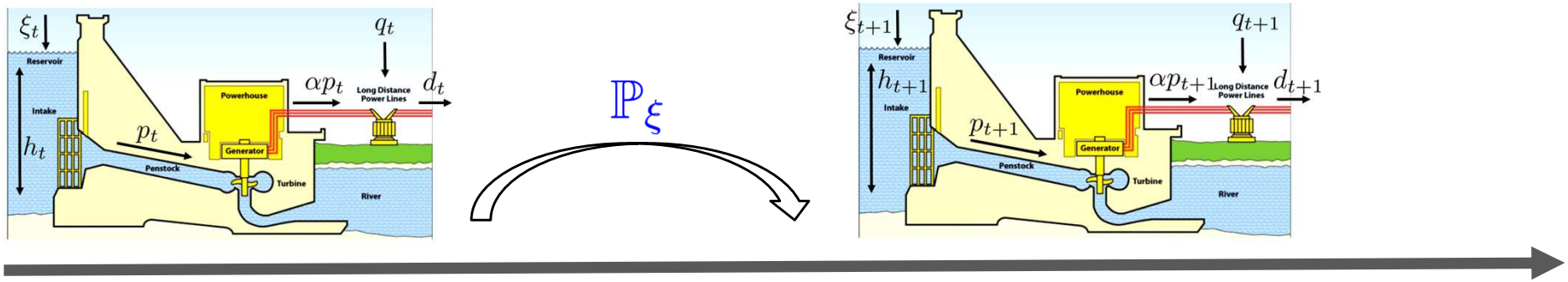
From MSP to MDP

CentraleSupélec

- **Motivations** for MDP:
  $\rightarrow$ very long horizon
  $\rightarrow$ interested in finding a **decision rule/policy** that depends on the observed system state
- An appropriate definition of **system state** is the central point in MDP; whereas states not necessarily appear explicitly in traditional MSP

State and policy in MSP

CentraleSupélec



$$\boldsymbol{s}_t = (h_t, d_t, c_t^e)$$

$$\boldsymbol{a}_t = (p_t, q_t)$$

$$\boldsymbol{s}_{t+1} = (h_{t+1} = h_t - p_t + \xi_{t+1}, d_{t+1}, c_{t+1}^e)$$

$$\boldsymbol{a}_{t+1} = (p_{t+1}, q_{t+1})$$

$$\begin{aligned} \min_{p_t, q_t} \quad & c_t^e q_t + c^p p_t + \mathcal{Q}(\boldsymbol{s}_t, \boldsymbol{a}_t) \\ \text{s.t.} \quad & \alpha p_t + q_t = d_t \\ & 0 \le p_t \le h_t \le h^{max}, \\ & p_t, q_t \ge 0 \end{aligned}$$

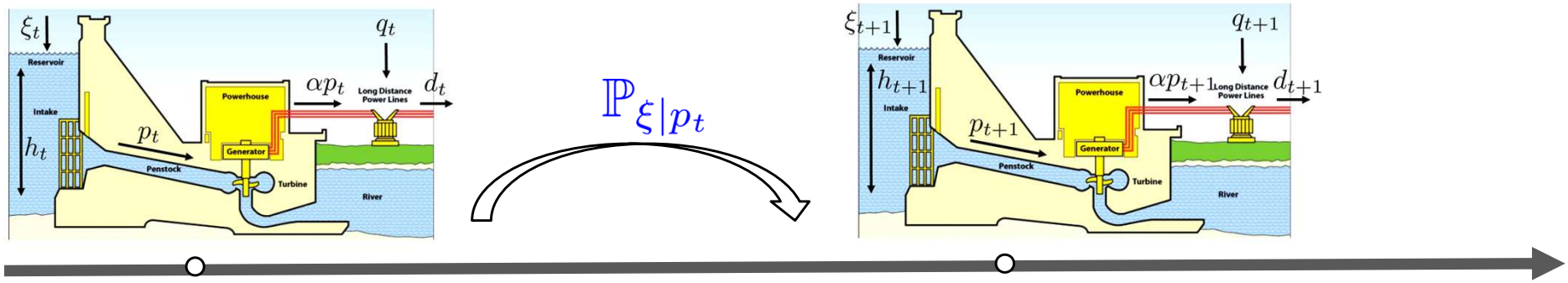
$$\begin{aligned} \min_{p_{t+1}, q_{t+1}} \quad & c_{t+1}^e q_{t+1} + c^p p_{t+1} + \mathcal{Q}(\boldsymbol{s}_{t+1}, \boldsymbol{a}_{t+1}) \\ \text{s.t.} \quad & \alpha p_{t+1} + q_{t+1} = d_{t+1} \\ & 0 \le p_{t+1} \le h_{t+1} \le h^{max}, \\ & p_{t+1}, q_{t+1} \ge 0 \end{aligned}$$

$$\text{Policy: } (h_t, d_t, c_t^e) \overset{\pi(\boldsymbol{a}_t|\boldsymbol{s}_t)}{\Longrightarrow} (p_t, q_t)$$

Difficulty with endogenous uncertainty

CentraleSupélec

- Exogenous vs. endogenous uncertainty
  - → **Exogenous**, e.g., inflows
  - → **Endogenous**, e.g., degradation rate of a robot car depends on the driving behaviors (fast, slow, etc.) ⇒ not easy modeled in MSP

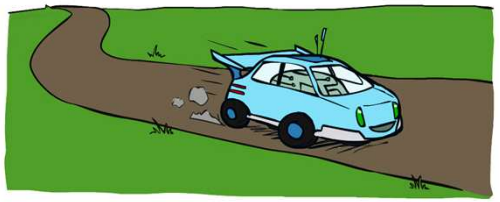


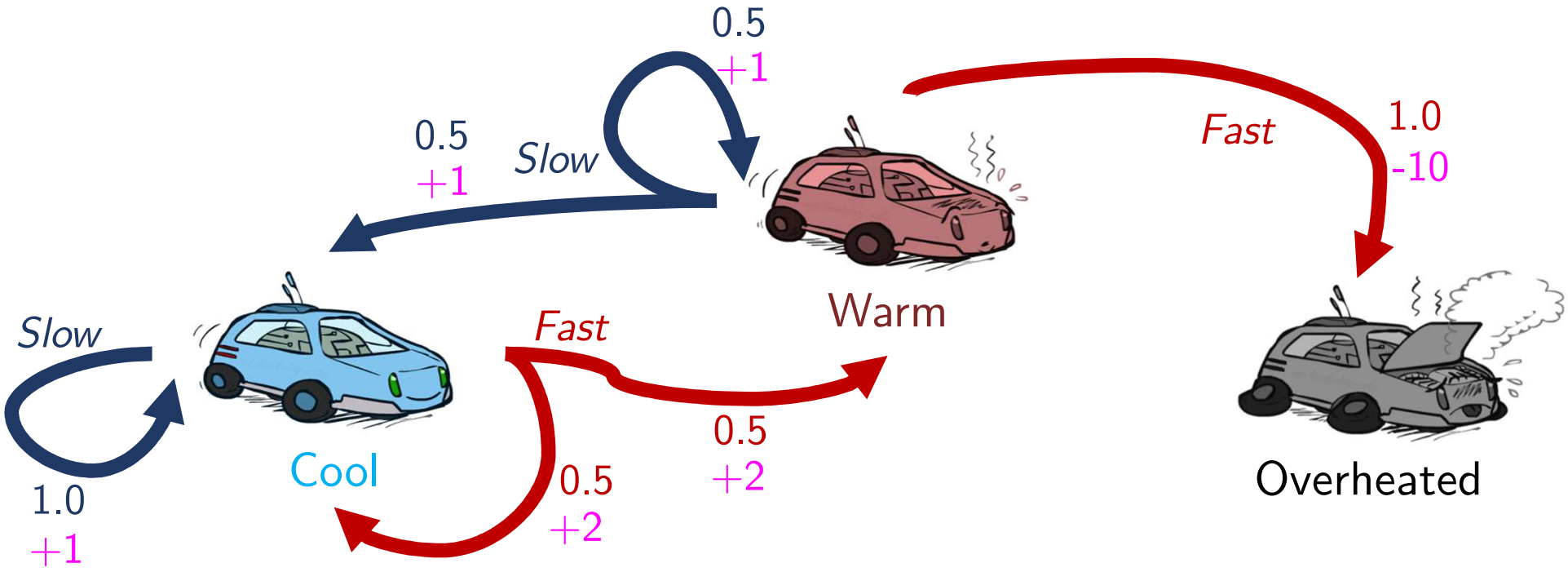
- MDP is a natural fit in this case, but at the sacrifice of assuming discrete (small) decision space (traditionally)

Typical MDP example: racing

CentraleSupélec

- A robot car wants to travel far, quickly
- 3 **states**: Cool, Warm, Overheated
- 2 **actions**: *Slow*, *Fast*
- Going faster gets double **reward** as long as not overheated
- **Goal**: maximize sum of rewards

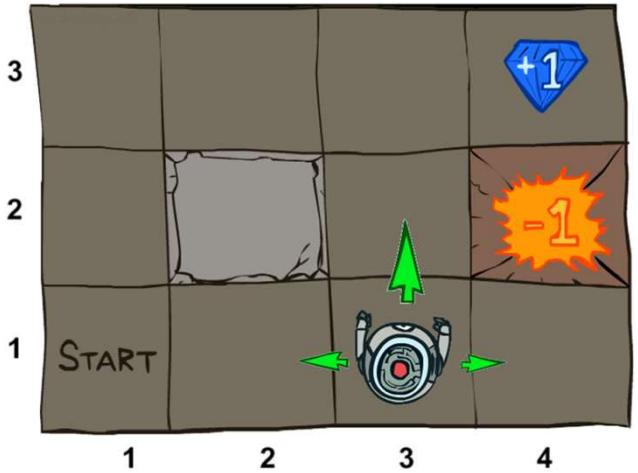




Example: grid world

CentraleSupélec

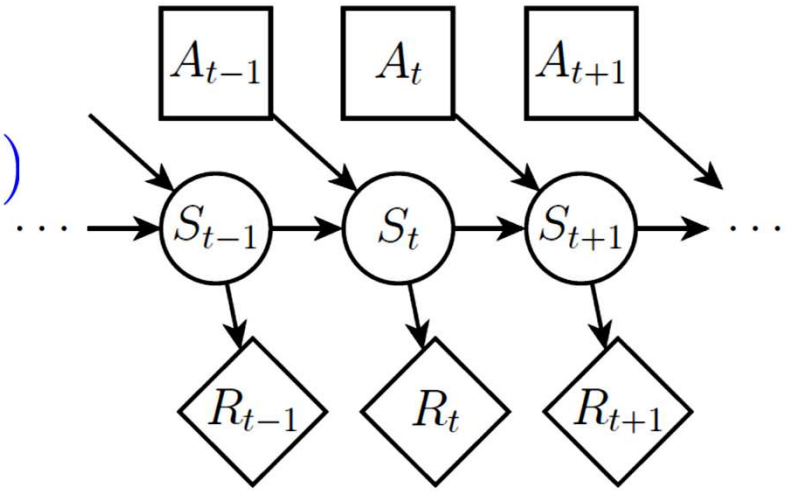
- A maze-like problem
  - → The agent lives in a grid
  - → Walls block the agent's path



- **Noisy movement:** actions do not always go as planned
  - → $p = 0.8$, the action North takes the agent North (if no wall there)
  - → $p = 0.1$, North takes the agent West; $p = 0.1$ East
  - → If there is a wall in the direction, the agent stays put
- The agent receives rewards each time step
  - → Small "living" reward each step (can be negative)
  - → Big rewards come at the end (good or bad)
- **Goal**: maximize sum of rewards

Markov decision processes

- A **MDP** is defined by
  - → a set of system states $s \in S$
  - → a set of actions/decisions, $a \in A$
  - → a transition function $T(s, a, s')$
    - probability that action a at state s leads to s', i.e. $P(s'|s, a)$
    - also called the dynamics, natural fit of endogenous uncertainty
  - → a reward function $R(s, a, s')$
    - sometimes just $R(a)$ or $R(s, a)$
  - → initial state is given



What is Markov about MDPs

CentraleSupélec

- **Markov**: given the present state, the future and the past are independent
- For MDPs, "Markov" means action outcomes depends only on the current state & action

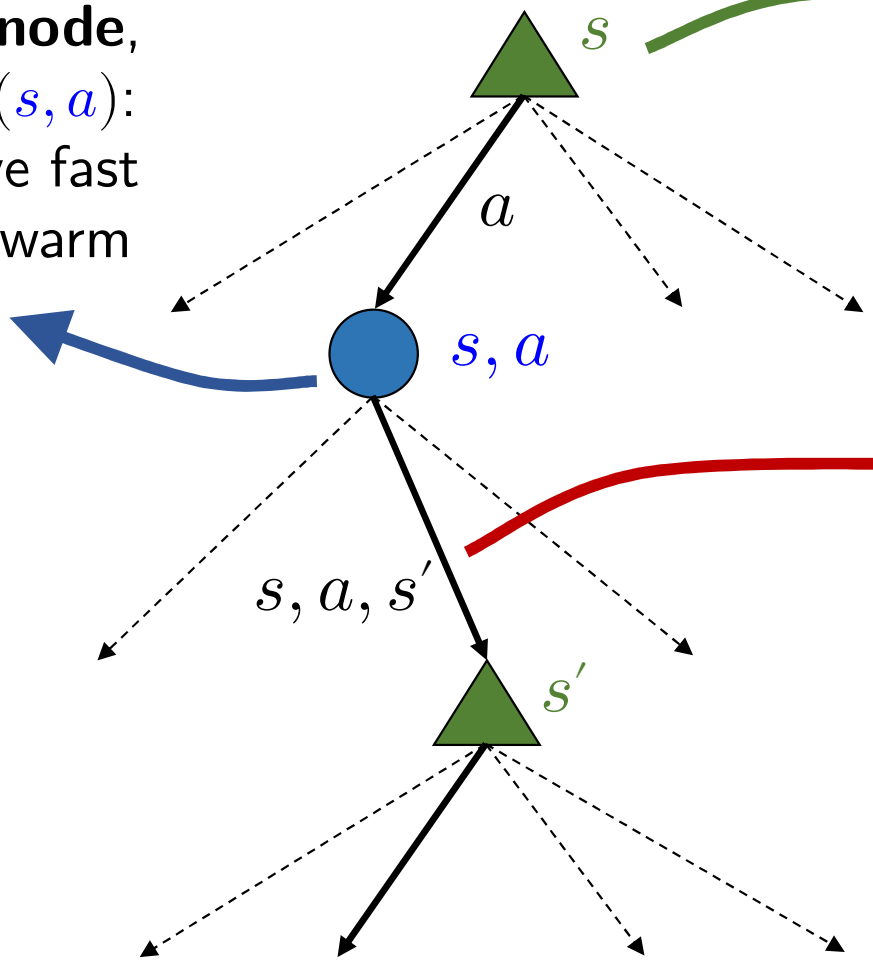
$$P(S_{t+1} = s' | S_t = s_t, A_t = a_t, S_{t-1} = s_{t-1}, A_{t-1}, \dots S_0 = s_0)$$

$$= P(S_{t+1} = s' | S_t = s_t, A_t = a_t)$$

MDP search tree

- Each MDP state projects a scenario search tree

chance node, Q-state (s, a): e.g., drive fast when in warm state



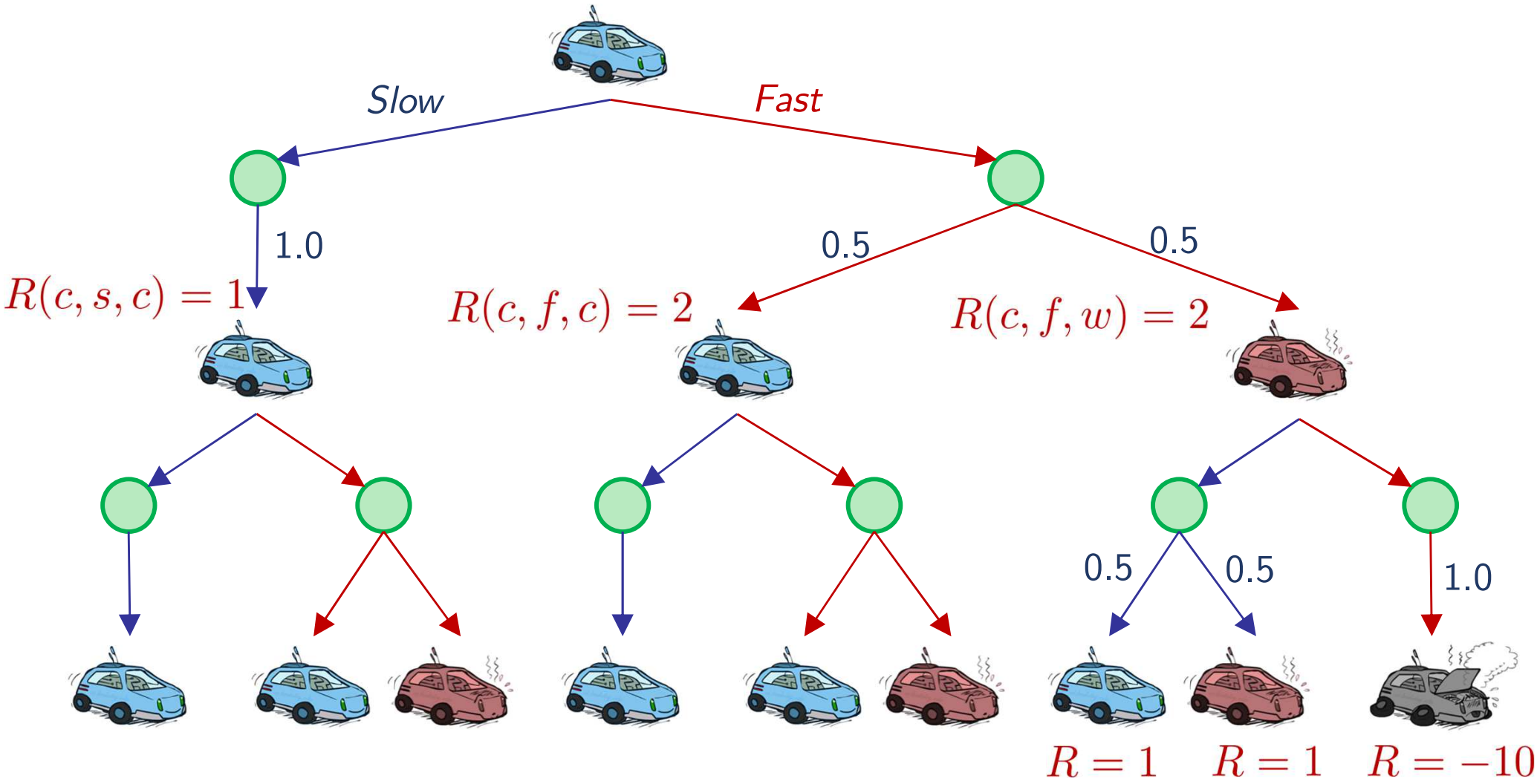
Warm

(s, a, s') called a **transition**

$T(s, a, s') = P(s'|s, a)$

$R(s, a, s')$

Racing search tree



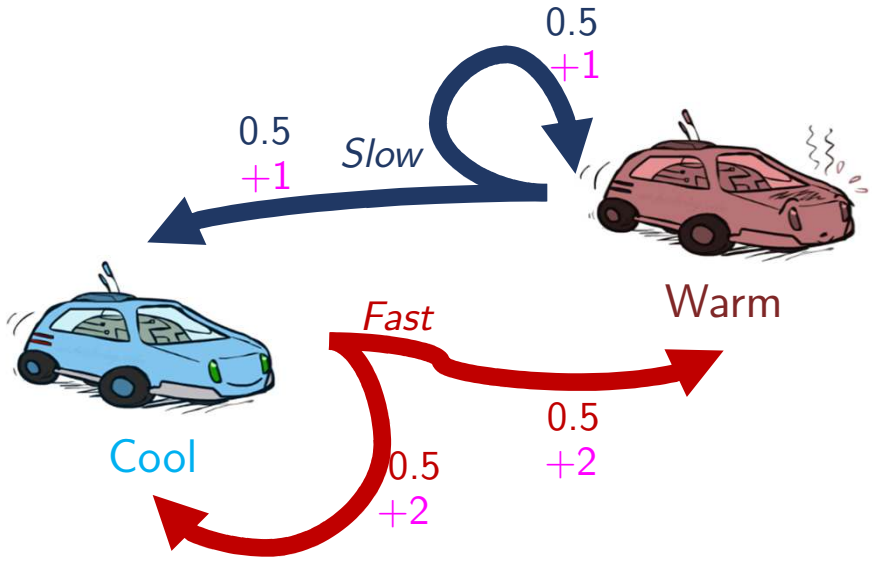
Policies

- In MSP, we want an optimal sequence of decisions, one for each node in the scenario tree, $\Xi \to A$
- For MDPs, we want an optimal policy $\pi^\star : S \to A$

  $\to$ a policy $\pi: S \to A$ gives an action for each state

  $\to$ optimal policy: maximizes **expected utility** if followed

  $\to$ an explicit policy defines a reflex agent: reduce to a Markov reward process

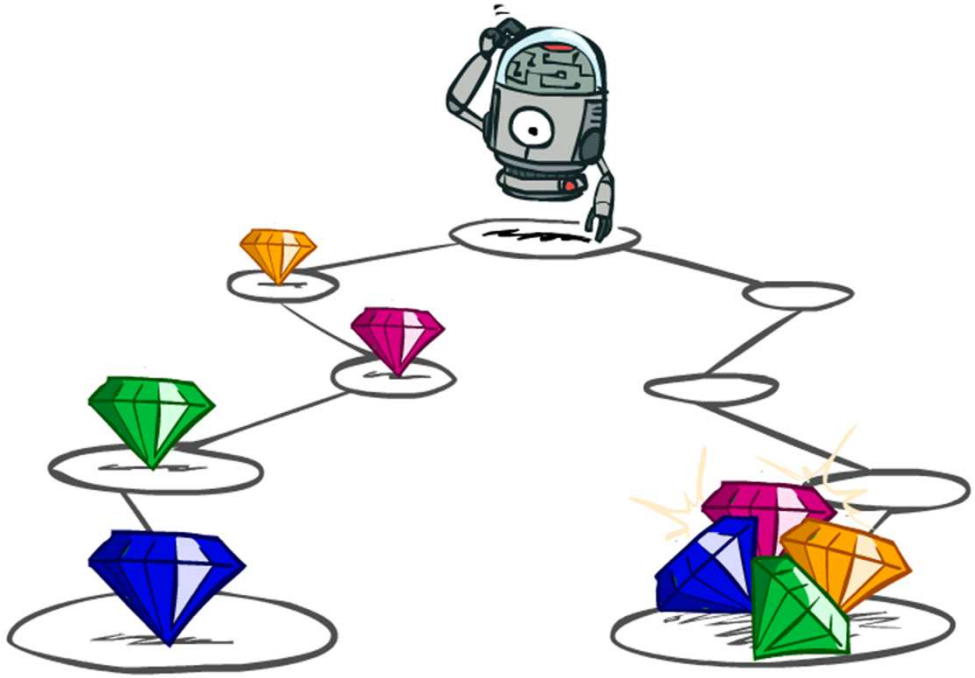


Utility of sequence

- What preferences should an agent have over reward sequences?
- More or less?

  [1, 2, 2] or [2, 3, 4]
- Now or later?

  [1, 0, 0] or [0, 0, 1]



Discounted utility

CentraleSupélec

- It's reasonable to maximize the sum of rewards of the sequence
- It's also reasonable to prefer rewards now to rewards later
- **One solution:** values of rewards decay exponentially

1

Worth now

γ

Worth next step

γ^2

Worth in 2 steps

$$U([r_0, r_1, r_2, \ldots]) = r_0 + \gamma r_1 + \gamma^2 r_2 \cdots$$

- **Example**: $\gamma = 0.5$

$\rightarrow U([1,2,3]) = 1\times1 + 0.5\times2 + 0.25\times3$

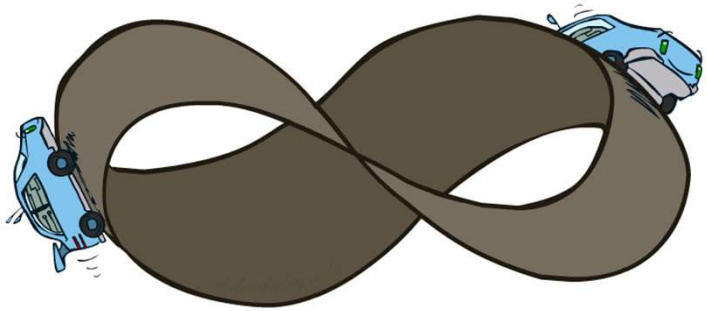
$\rightarrow U([1,2,3]) < U([3,2,1])$

Infinite horizon

CentraleSupélec

- What if the horizon is infinite?

  $\rightarrow$ infinite rewards?



- Solutions

  $\rightarrow$ use **discounted utility** $0 < \gamma < 1$

$$U([r_0, \dots r_\infty]) = \sum_{t=0}^{\infty} \gamma^t r_t \leq r_{\max}/(1-\gamma)$$

  $\rightarrow$ smaller γ means smaller "horizon", shorter term focus

  $\rightarrow$ **absorbing state**: guarantee that for every policy, a terminal state will eventually be reached

Value function and optimal policy

CentraleSupélec

- A value function for a policy π, written $V^\pi : S \to \mathbf{R}$ gives the expected utility (sum of discounted rewards) when acting under that policy

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) | s_0 = s, a_t = \pi(s_t)\right]$$

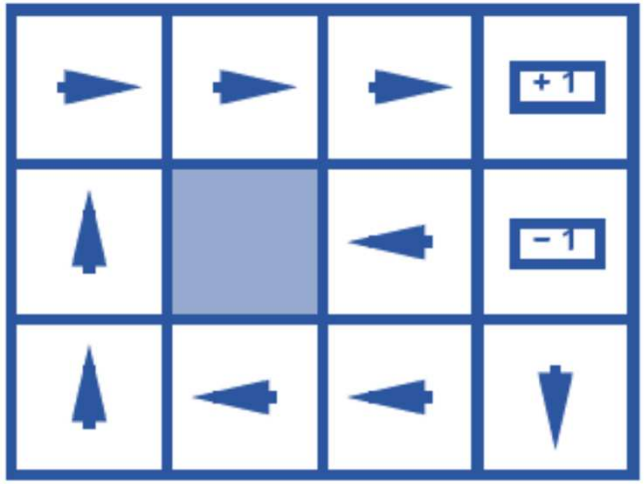
- The optimal policy is the policy that achieves the highest value for every state

$$\pi^\star = \arg\max_\pi V^\pi(s)$$

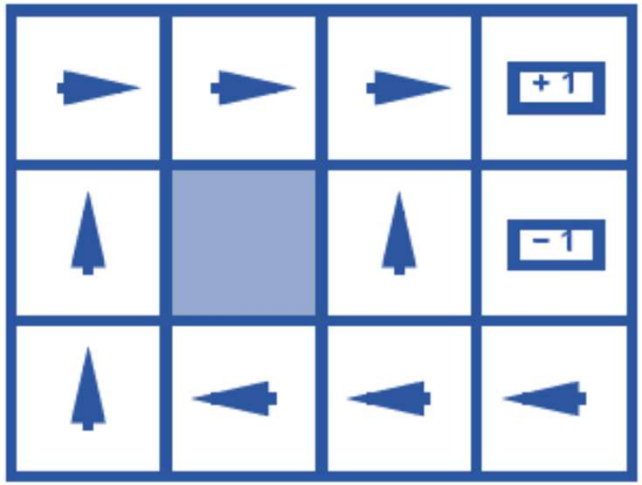
- and its optimal value is written $V^\star(s)$

Optimal policy

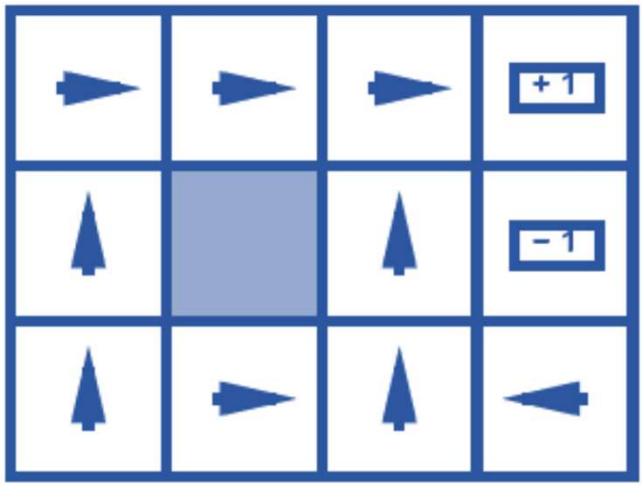
CentraleSupélec



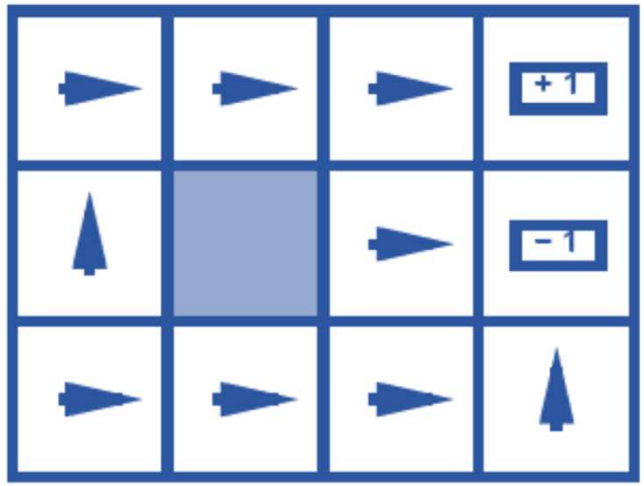
$R(s) = -0.01$



$R(s) = -0.03$



$R(s) = -0.4$



$R(s) = -2.0$

Recap: defining MDPs

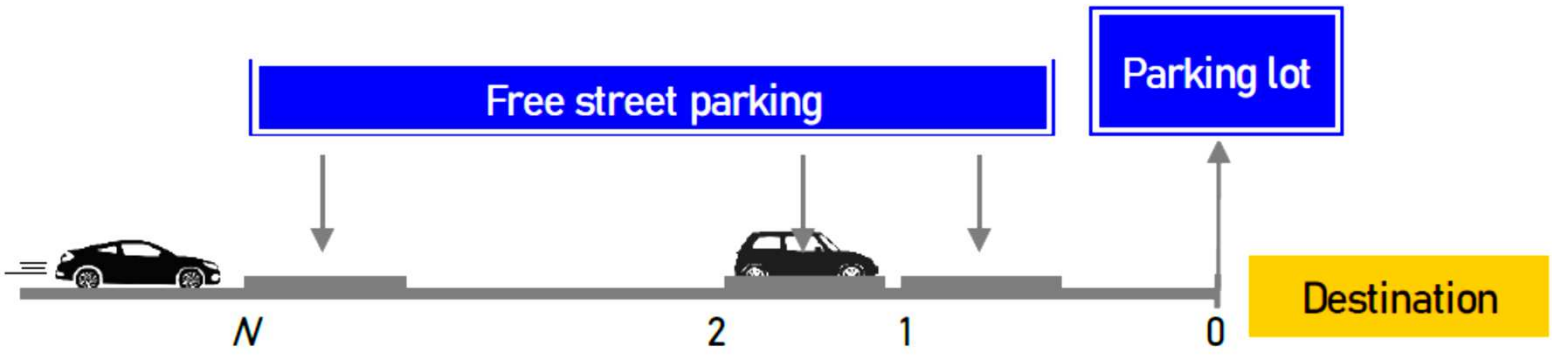
CentraleSupélec

- Markov decision processes
  - → System states $s \in S$
  - → Start state s_0
  - → Possible actions $a \in A$
  - → Transitions $T(s, a, s') = P(s'|s, a)$
  - → Rewards $R(s, a, s')$ and discount γ
- MDP quantities so far
  - → utility: sum of (discounted) rewards
  - → policy: choice of action for each state
  - → value function for a given policy: expected utility when acting under that policy
  - → optimal policy: gives the highest value for every state

Tutorial: parking problem

CentraleSupélec

- You are driving to a restaurant in Paris
- start from street parking spot N, and you can observe whether parking spot $n, n \in \{N, N-1, \dots, 0\}$ is empty only when arriving at the spot.
- by p, a spot is empty; $1-p$, a spot is occupied.
- the parking lot at the restaurant is always available with a high fee c.
- The cost of parking at a street spot n is equal to the distance from the restaurant n



CentraleSupélec

MDP formulation

- States

$$s \in S \cup \{(0,1), \Delta\}, \text{ where } S = \mathbf{Z}_+ \times \{0,1\}$$

- Actions

$$A_s = \begin{cases} \{C\} & s = (n,0) \\ \{C,P\} & s = (n,1) \\ \{P\} & s = (0,1) \end{cases}$$

- Costs (rewards)

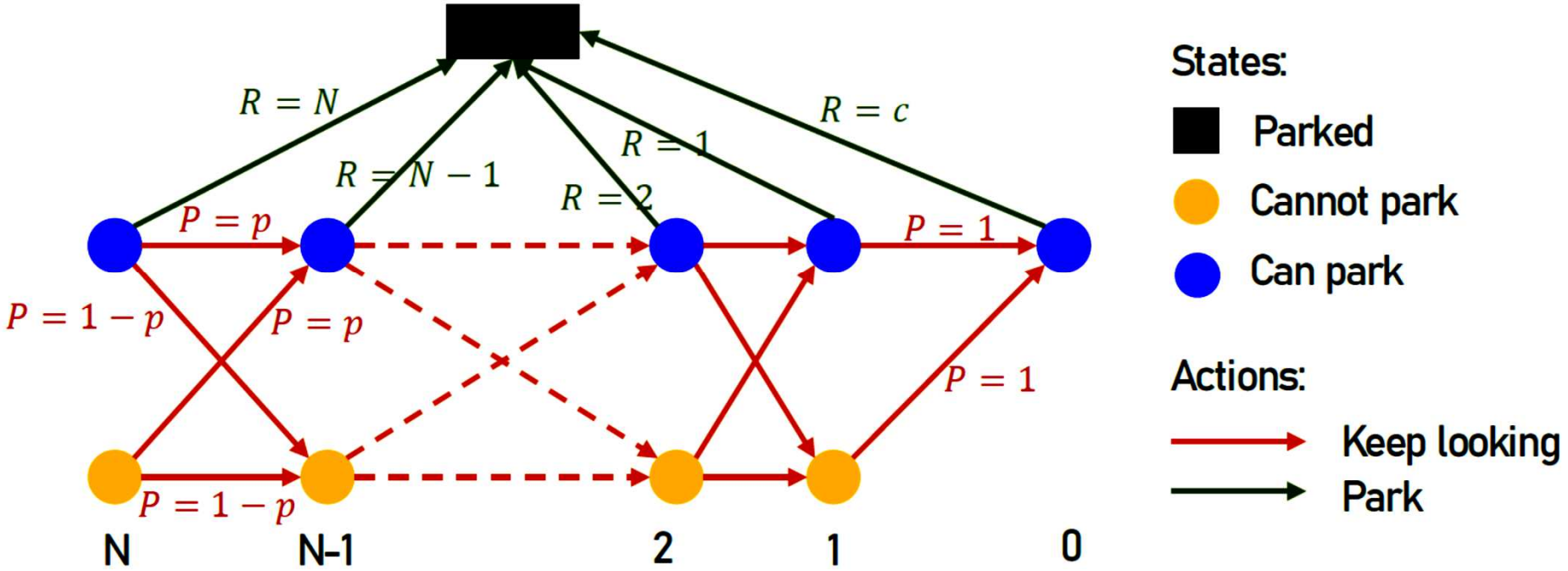
$$R_n(s,a) = \begin{cases} 0 & s \in S, a = C \\ n & s \in S, a = P \\ c & s = (0,1), a = P \end{cases}$$

CentraleSupélec

MDP formulation

- Transition probabilities

$$T((n,\cdot),C,(n-1,1)) = p,\ \forall n \geq 2$$
$$T((n,\cdot),C,(n-1,0)) = 1-p,\ \forall n \geq 2$$
$$T((1,\cdot),C,(0,1)) = 1$$
$$T((n,1),P,\Delta) = 1$$



MSP formulation

CentraleSupélec

- Indicating parameter

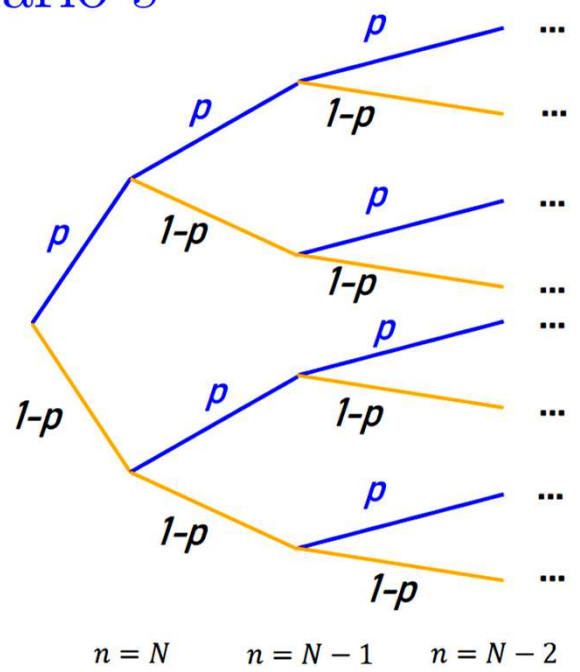
$$\delta_{n,s} = \begin{cases} 0 & \text{if spot } n \text{ is occupied in scenario } s \\ 1 & \text{if spot } n \text{ is empty in scenario } s \end{cases}$$

- Binary decision variable

$$y_{n,s} = \begin{cases} 0 & \text{if do not park in spot } n \text{ in scenario } s \\ 1 & \text{if park in spot } n \text{ in scenario } s \end{cases}$$

- Probability of scenario s

$$p_s = \prod_{n=1}^{N} (p\delta_{n,s} + (1-p)(1-\delta_{n,s}))$$



MSP formulation

CentraleSupélec

- MILP formulation

$$\min_{y_{n,s}} \quad \sum_s p_s c_s$$

$$\text{s.t.} \quad c_s = \sum_{n=1}^{N} y_{n,s} n + (1 - \sum_{n=1}^{N} y_{n,s}) c, \ \forall s \in S$$

$$y_{n,s} \leq \delta_{n,s}, \ \forall n = 1, ..., N, s \in S$$

$$\sum_{n=1}^{N} y_{n,s} \leq 1, \ \forall s \in S$$

$$y_{n,s} = y_{n,s'}, \ \forall \, N_{s,s'}^{parent} \leq n \leq N, s, s' \in S$$

MSP formulation

CentraleSupélec

- For example: $N = 3, p = 0.6, c = 4$

$$\delta_{n,s} = \begin{cases} 0 & \text{if spot } n \text{ is occupied in scenario } s \\ 1 & \text{if spot } n \text{ is empty in scenario } s \end{cases}$$

$$y_{n,s} = \begin{cases} 0 & \text{if do not park in spot } n \text{ in scenario } s \\ 1 & \text{if park in spot } n \text{ in scenario } s \end{cases}$$

	$s=1$		$s=2$		$s=3$		$s=4$		$s=5$		$s=6$		$s=7$		$s=8$	
n	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$	$\delta_{n,s}$	$y_{n,s}$
1	0	0	1	1	0	0	1	0	0	0	1	1	0	0	1	0
2	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
3	0	0	0	0	0	0	0	0	1	0	1	0	1	0	1	0

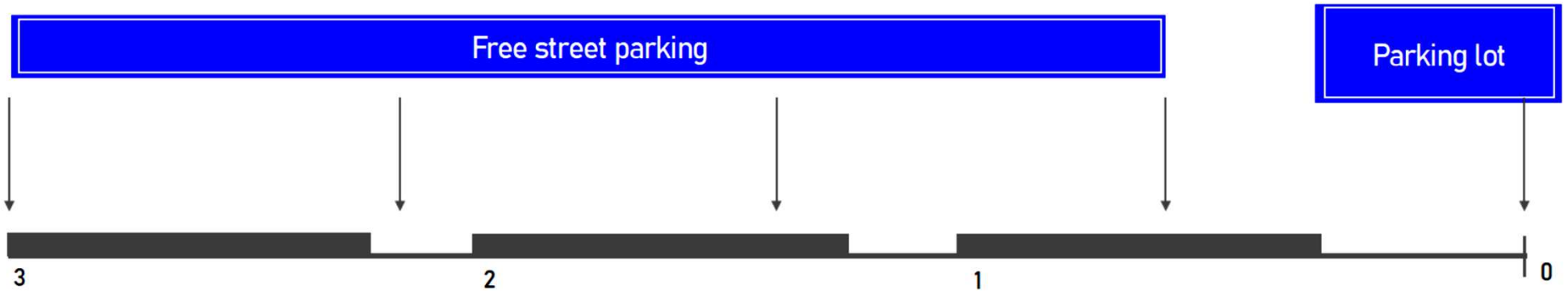
MDP solution

CentraleSupélec

$$V[(n,0)] = pV[(n-1,1)] + (1-p)V[(n-1,0)]$$

$$V[(n,1)] = \min\left\{ \underbrace{n}_{\text{To park}}, \underbrace{pV[(n-1,1)] + (1-p)V[(n-1,0)]}_{\text{To keep looking}} \right\}$$

$$N = 3, p = 0.6, c = 4$$



$$V[(3,0)] = 0.6V[(2,1)] + 0.4V[(2,0)]$$
$$= 0.6 \times 2 + 0.4 \times 2.2 = 2.08$$
$$V[(3,1)] = \min\{3, 2.08\} = 2.08$$

To keep looking

$$V[(2,0)] = 0.6V[(1,1)] + 0.4V[(1,0)]$$
$$= 0.6 \times 1 + 0.4 \times 4 = 2.2$$
$$V[(2,1)] = \min\{2, 2.2\} = 2$$

To park

$$V[(1,0)] = c = 4$$
$$V[(1,1)] = \min\{1, 4\} = 1$$

To park

MDP vs MSP

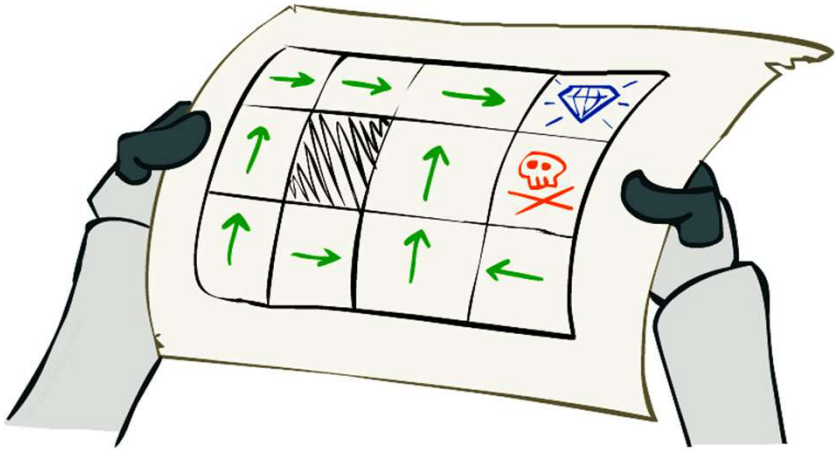
CentraleSupélec

	MSP	MDP
Uncertainty	• Can model non-Markovian • Typically exogenous • Modeled as scenario tree	• Markovian • Naturally endogenous • Transition dynamics
Time horizon	• Typically finite	• Finite or infinite
State	• Often not explicitly defined	• Explicit state definition
Decision/ action	• Continuous or discrete • Naturally handle complex constraints	• Discrete (discretization or approximation if continuous) • Limited in handling constraints
Solution	• Decisions per scenario	• Policy mapping state $\rightarrow$ action
Complexity	• Curse of dimensionality • Approximate (e.g., decision rules)	• Curse of dimensionality • Approximate (e.g., value function approximation)

Solving MDPs

CentraleSupélec

- How to find the optimal policy $\pi^\star : S \to A$?



- Model-based methods:
  1) Value iteration
  2) Policy iteration
  3) Linear programming for MDPs
- Model-free (sampling): reinforcement learning

Optimal quantities

CentraleSupélec

- The optimal value of a state s

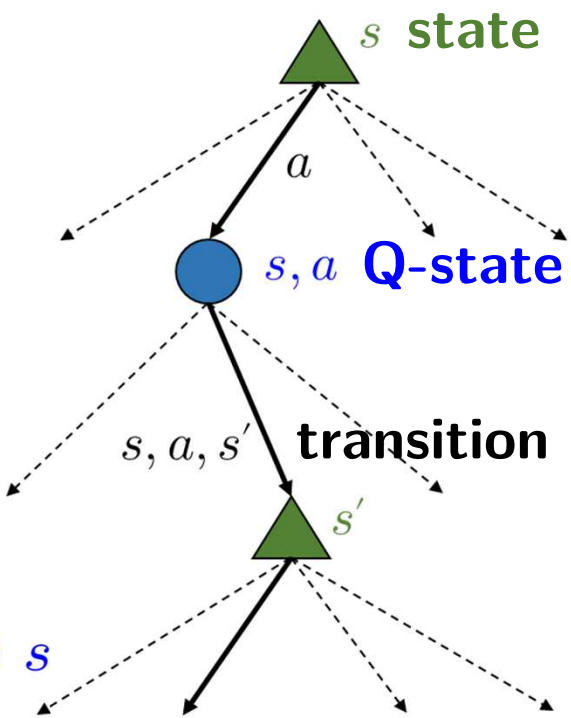
  $V^\star(s) =$ expected utility starting in s and acting optimally

- The optimal value of a Q-state (s, a)

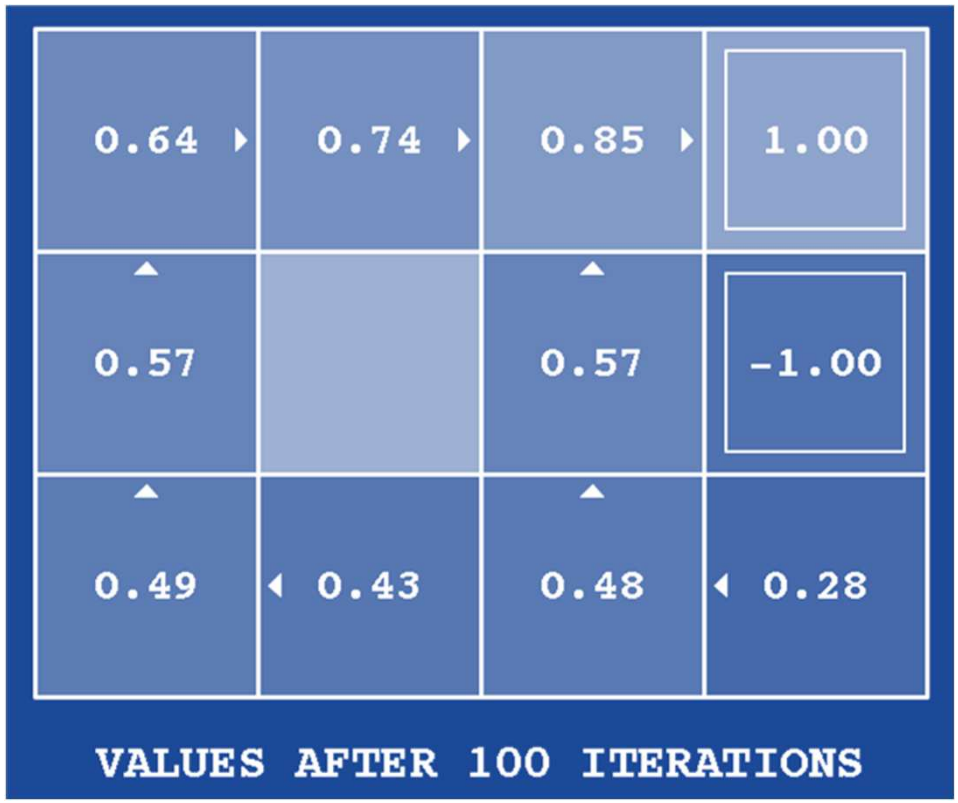
  $Q^\star(s, a) =$ expected utility starting out having taken action a from state s and (thereafter) acting optimally

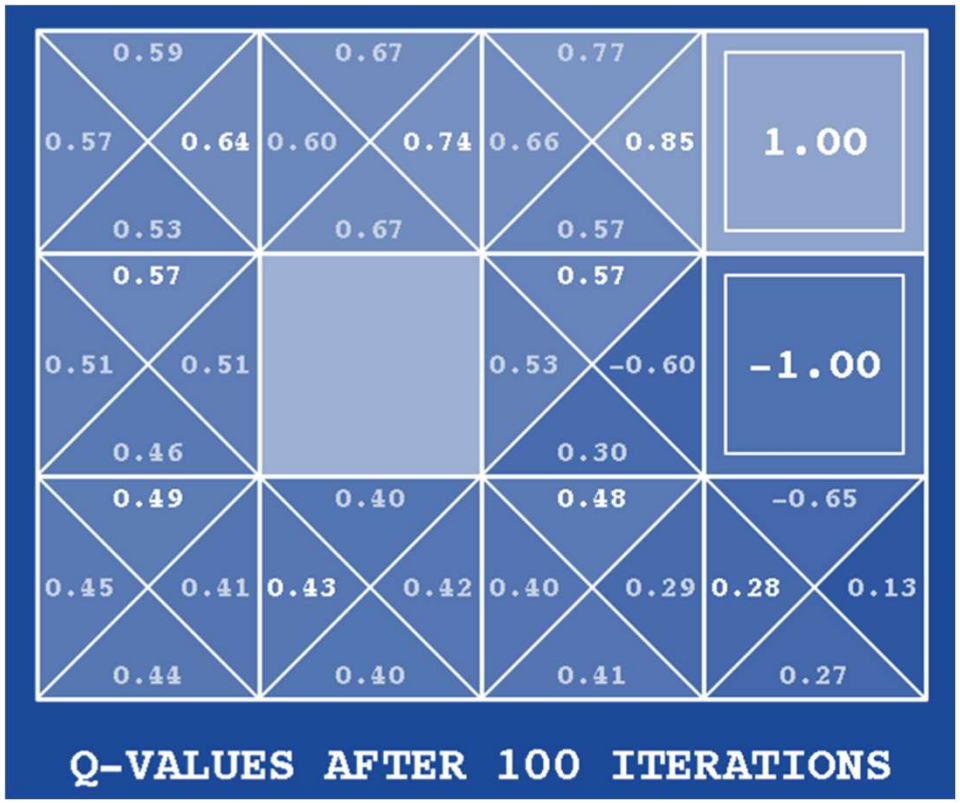
- The optimal policy

  $\pi^\star(s) =$ optimal action from state s



V* and Q*





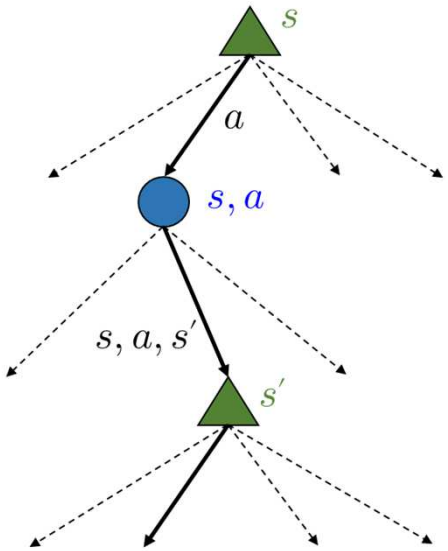
Noise = 0.2

Discount = 0.9

Living reward = 0

Values of states

- **Fundamental operation**: compute the (expected max) optimal value of a state
  - $\rightarrow$ Expected utility under optimal action
  - $\rightarrow$ Average sum of (discounted) rewards



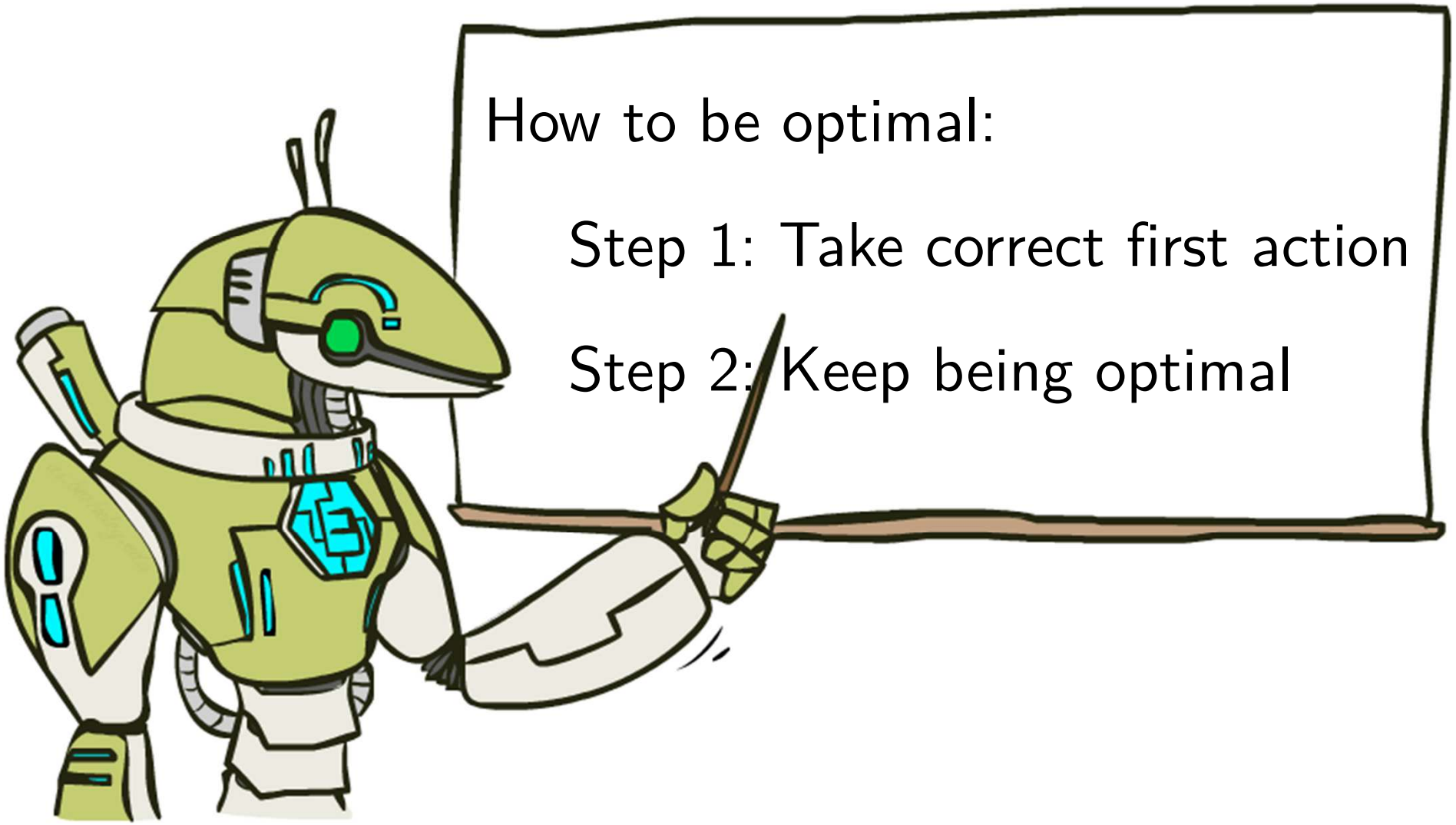
- **Recursive definition of values (Bellman optimality equations)**

$$V^\star(s) = \max_{a \in A} Q^\star(s, a)$$

$$Q^\star(s, a) = \sum_{s' \in S} T(s, a, s')[R(s, a, s') + \gamma V^\star(s')]$$

$$V^\star(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s')[R(s, a, s') + \gamma V^\star(s')]$$

The Bellman equations

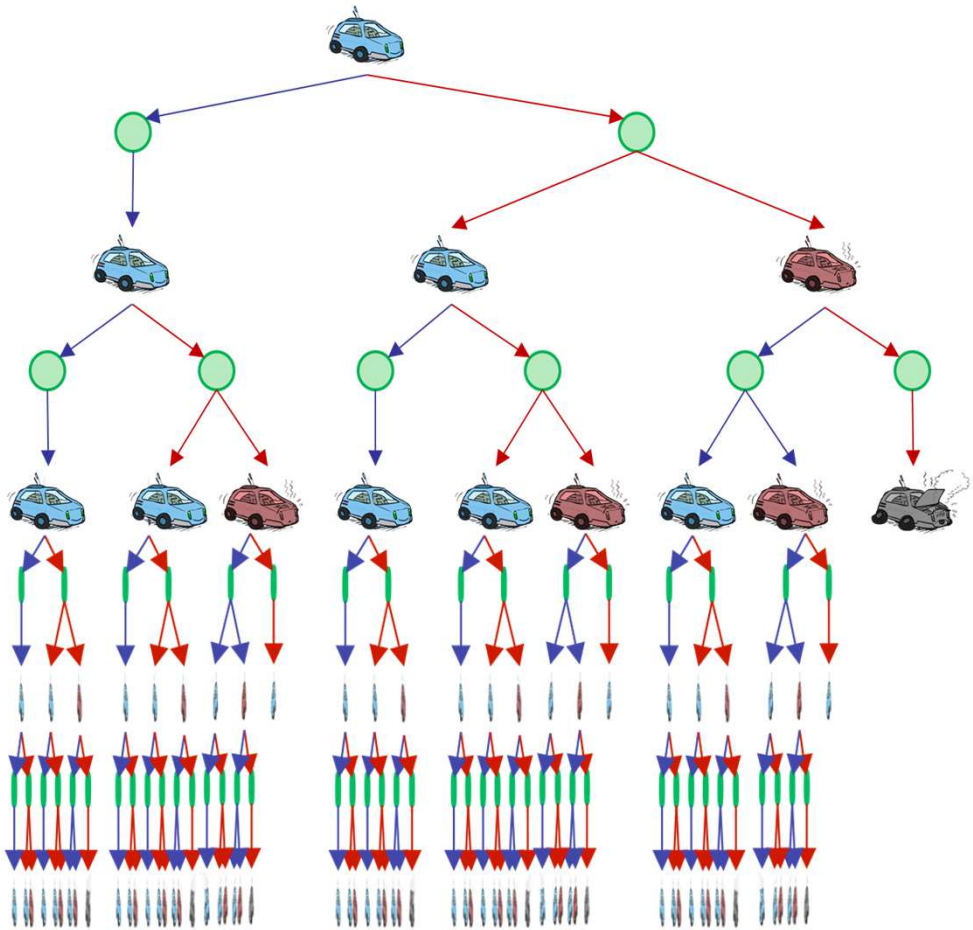


Time limited value

- Define $V_k(s)$ to be the optimal value of s if the decision horizon ends in k more steps
- How about infinite horizon (tree goes forever)?

  $\rightarrow$ idea: do a depth-limited computation, but with increasing depths until change is small

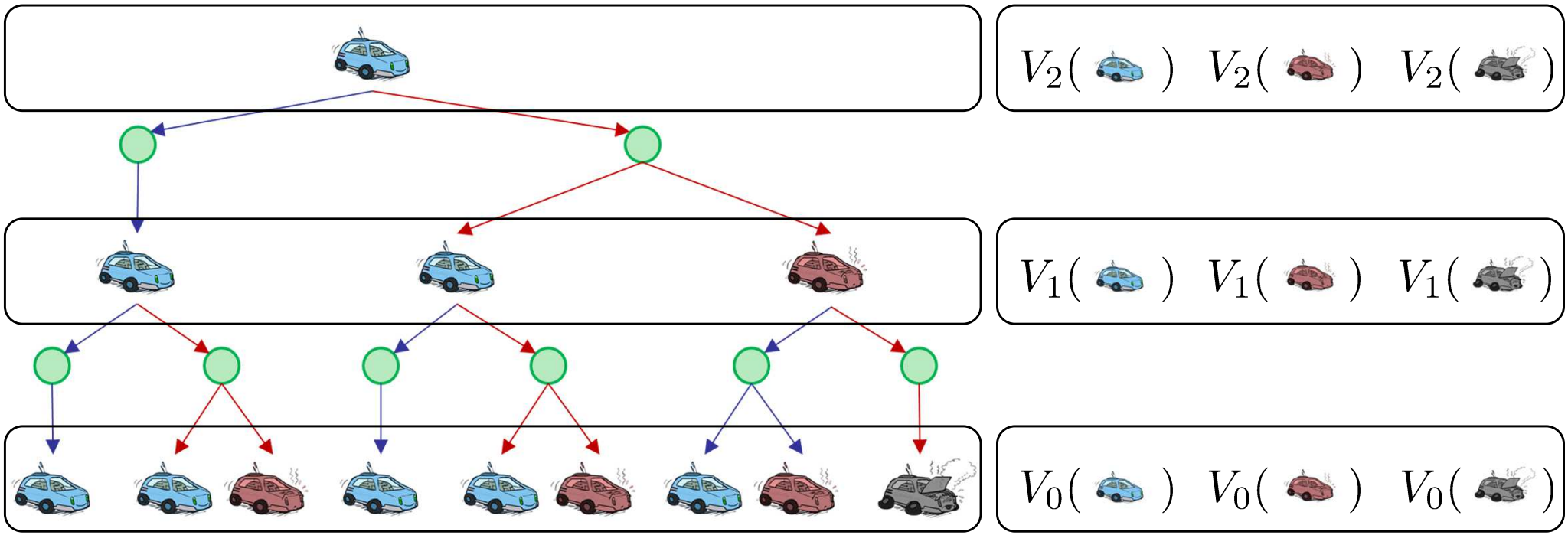
  $\rightarrow$ note: deep parts of the tree eventually don't matter if $\gamma < 1$



Time limited value and value iteration

CentraleSupélec

Decision horizon $K = 2$



$$V^\star(s) = \max_{a \in A} \sum_{s' \in S} T(s, a, s')[R(s, a, s') + \gamma V^\star(s')]$$

Value iteration

CentraleSupélec

- **Idea:** repeatedly update an estimate of the optimal value function according to Bellman optimality equation

- **Procedure**

  1) Initialize $V_0(s) \leftarrow 0,\ \forall s \in S$: no time steps left means an expected reward sum of zero

  2) Update:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} T(s,a,s')[R(s,a,s') + \gamma V_k(s')]$$

  3) Repeat until convergence

- Complexity of each iteration: $O(|S|^2|A|)$

Illustration of value iteration

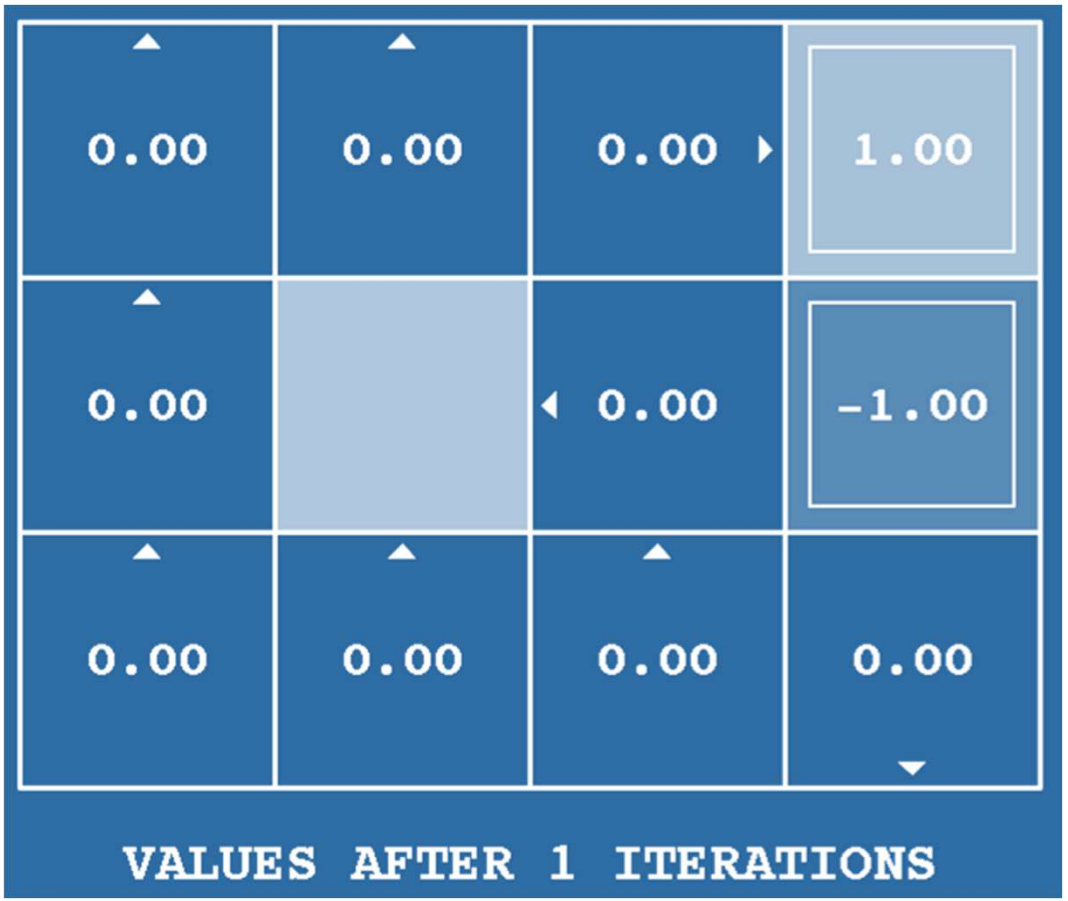
$k = 0$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration



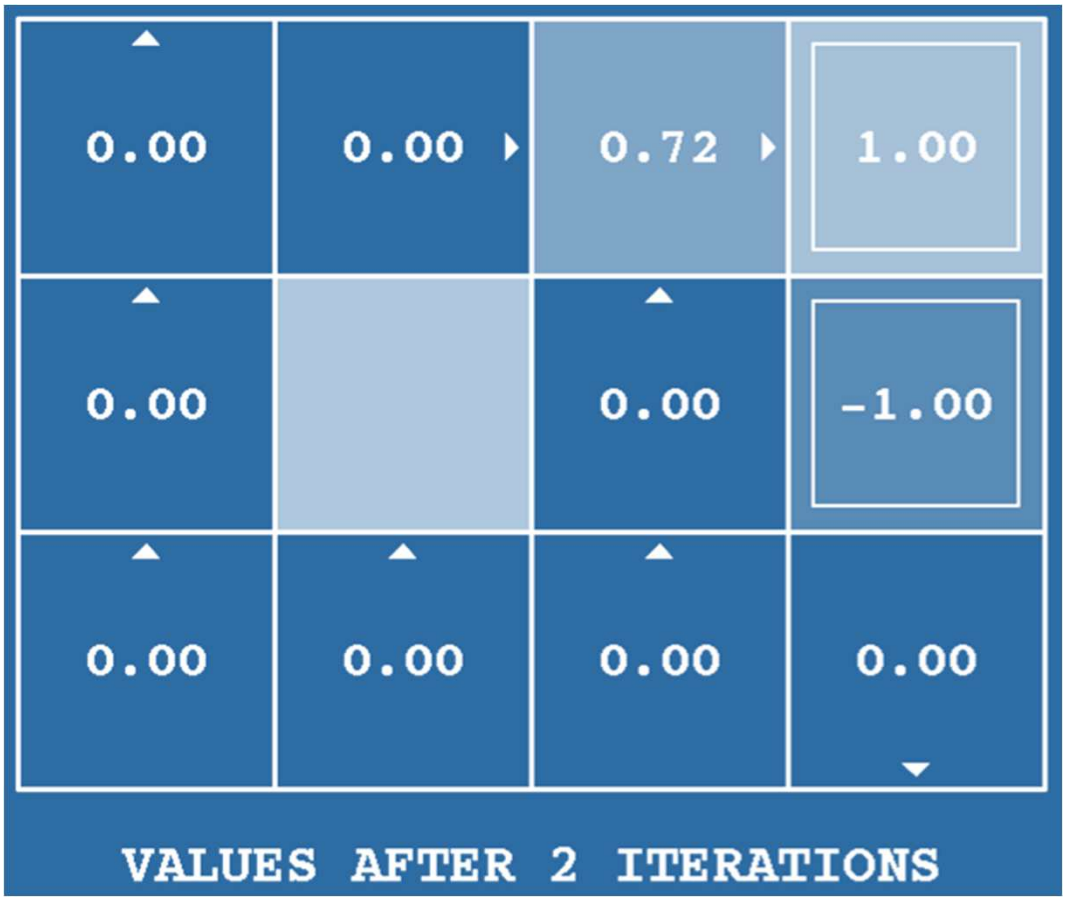
$k = 1$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration



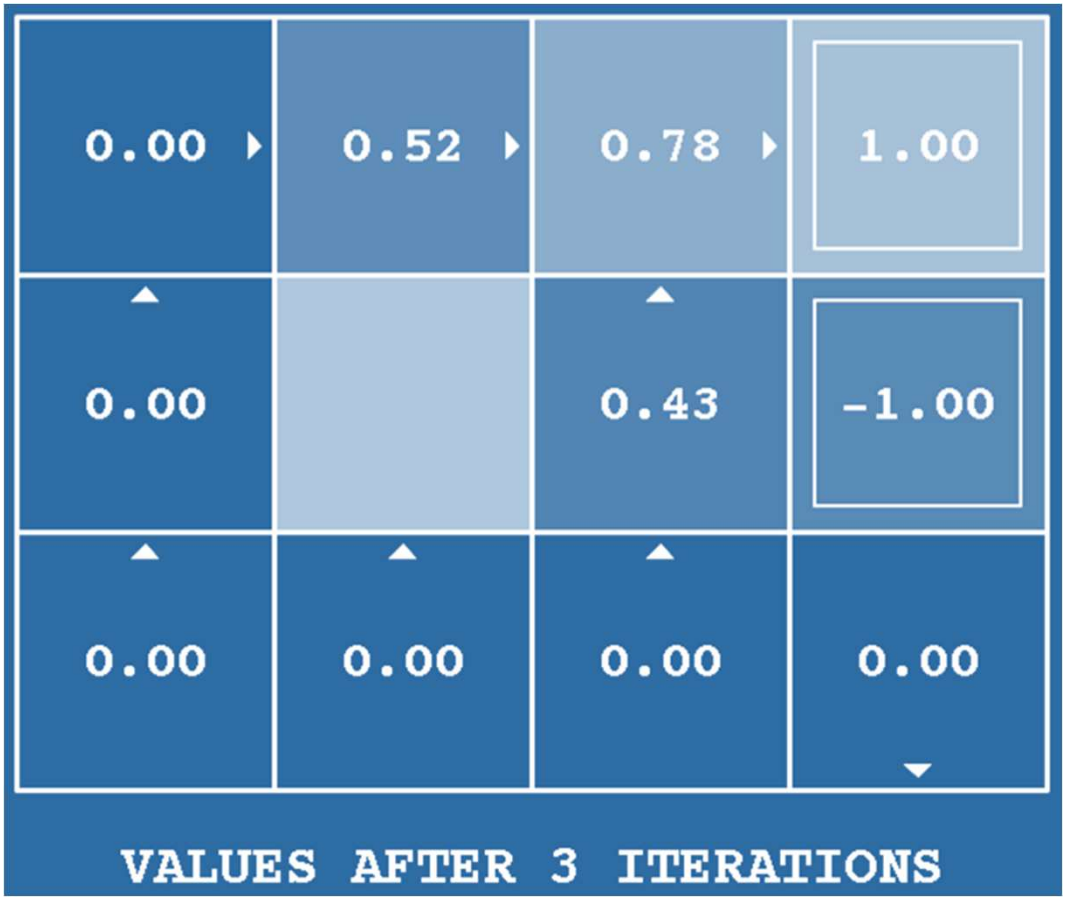
$k = 2$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration



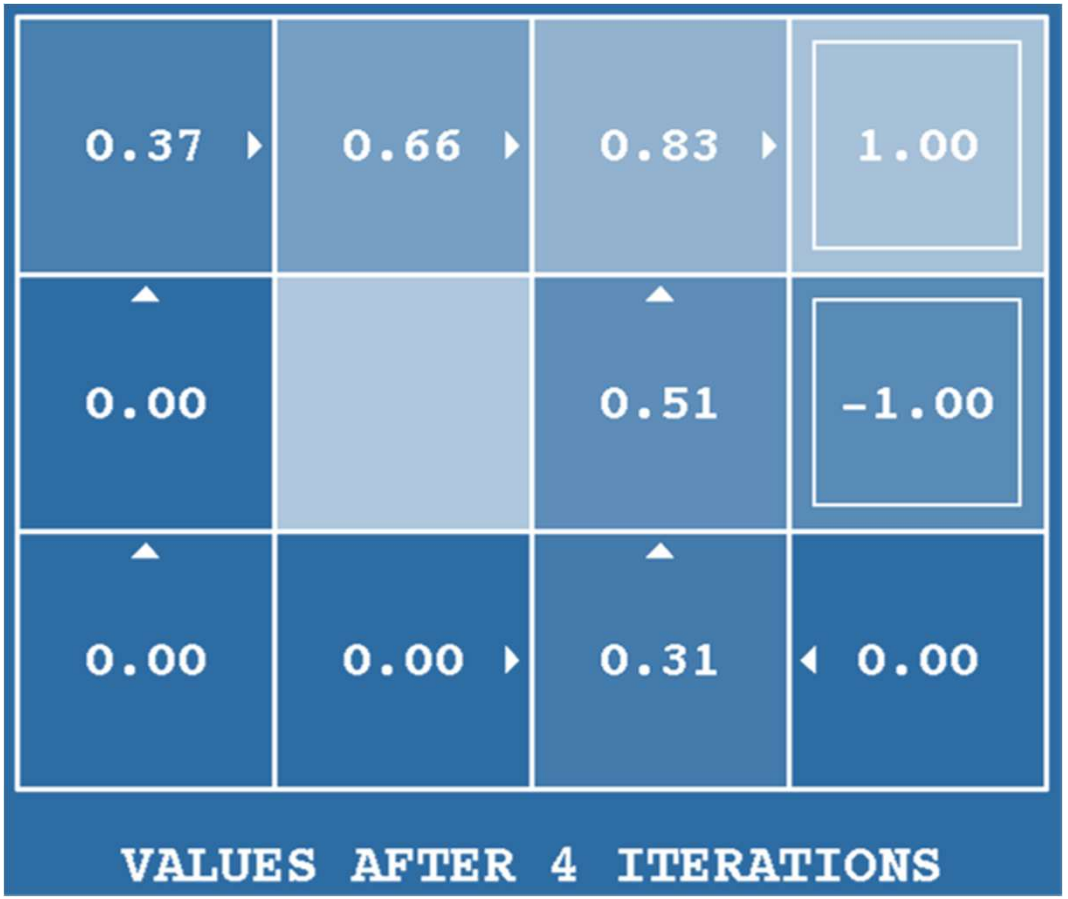
$k = 3$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration



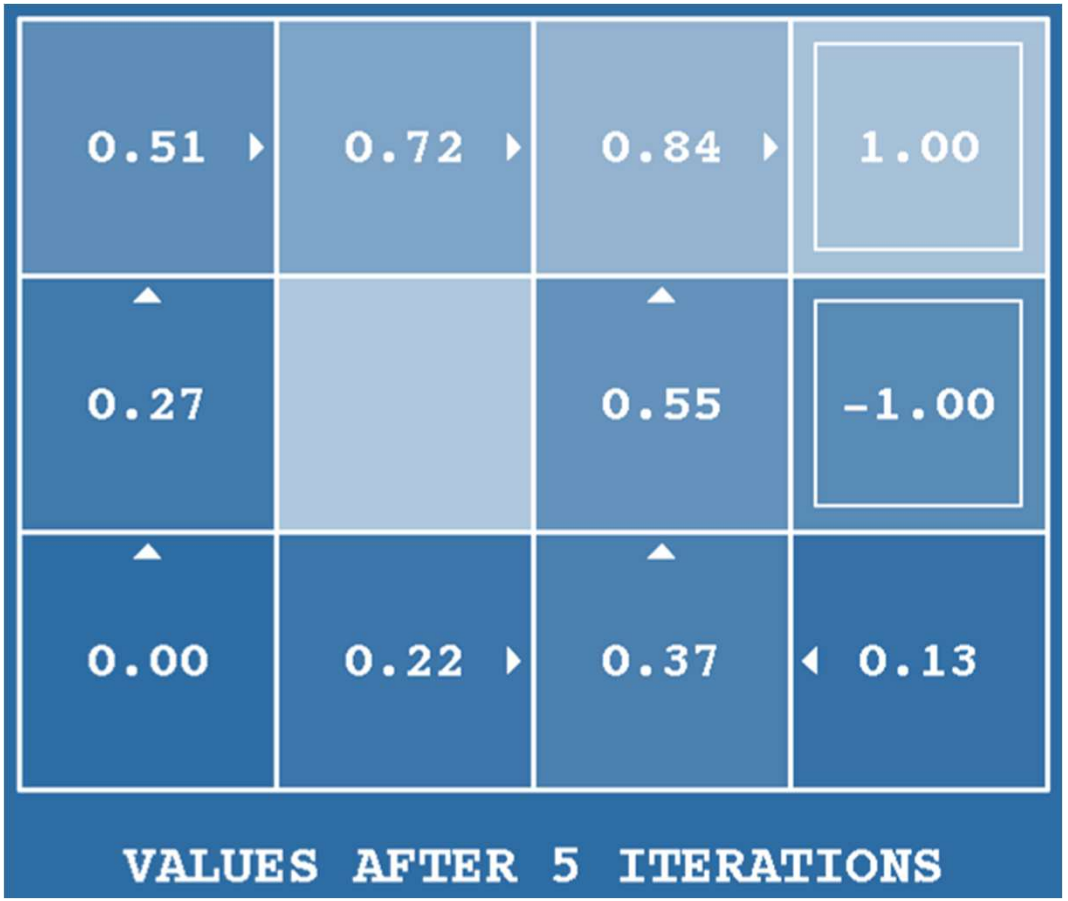
$k = 4$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration



$k = 5$

Noise $= 0.2$

Discount $= 0.9$

Living reward $= 0$

Illustration of value iteration

$k = 10$

Noise = 0.2

Discount = 0.9

Living reward = 0

Illustration of value iteration

$k = 100$

Noise $= 0.2$

Discount $= 0.9$

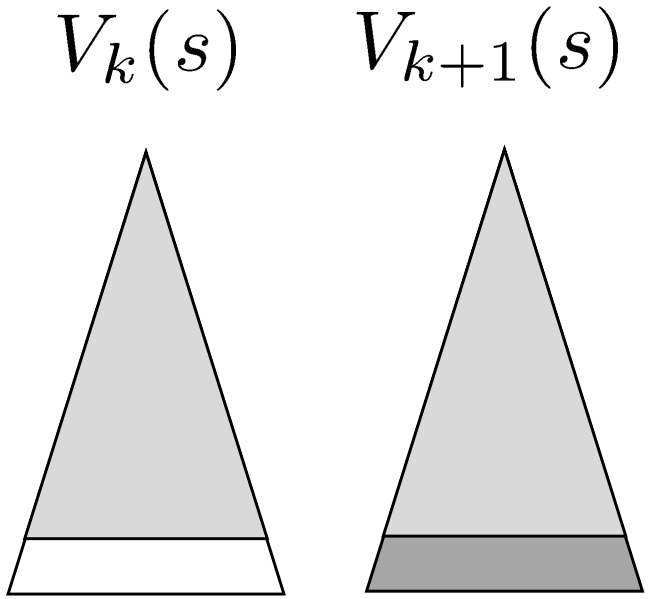
Living reward $= 0$

Convergence of value iteration

CentraleSupélec

Theorem: value iteration asymptotically converges to optimal value $V_k \rightarrow V^\star$

Conceptual proof:



- **Case 1**: if the decision horizon is finite with T steps, then, V_T holds the actual un-truncated values
- **Case 2**: if infinite horizon with $\gamma < 1$
- **Sketch**: for any state V_k and V_{k+1} can be viewed as depth $k+1$ expected-max results in nearly identical search trees
- The difference: on the bottom layer, V_{k+1} has actual rewards while V_k has zeros
- That last layer is at best all R_{MAX}, at worst R_{MIN}; but everything is discounted by γ^k that far out, so

$$|V_{k+1} - V_k| \leq \gamma^k R_{MAX}$$

Exercises

CentraleSupélec

- Implementing the MSP formulation for the parking problem
- **Questions**: In the parking problem, is it possible to simplify the MSP model? How?

yiping.fang@centralesupelec.fr